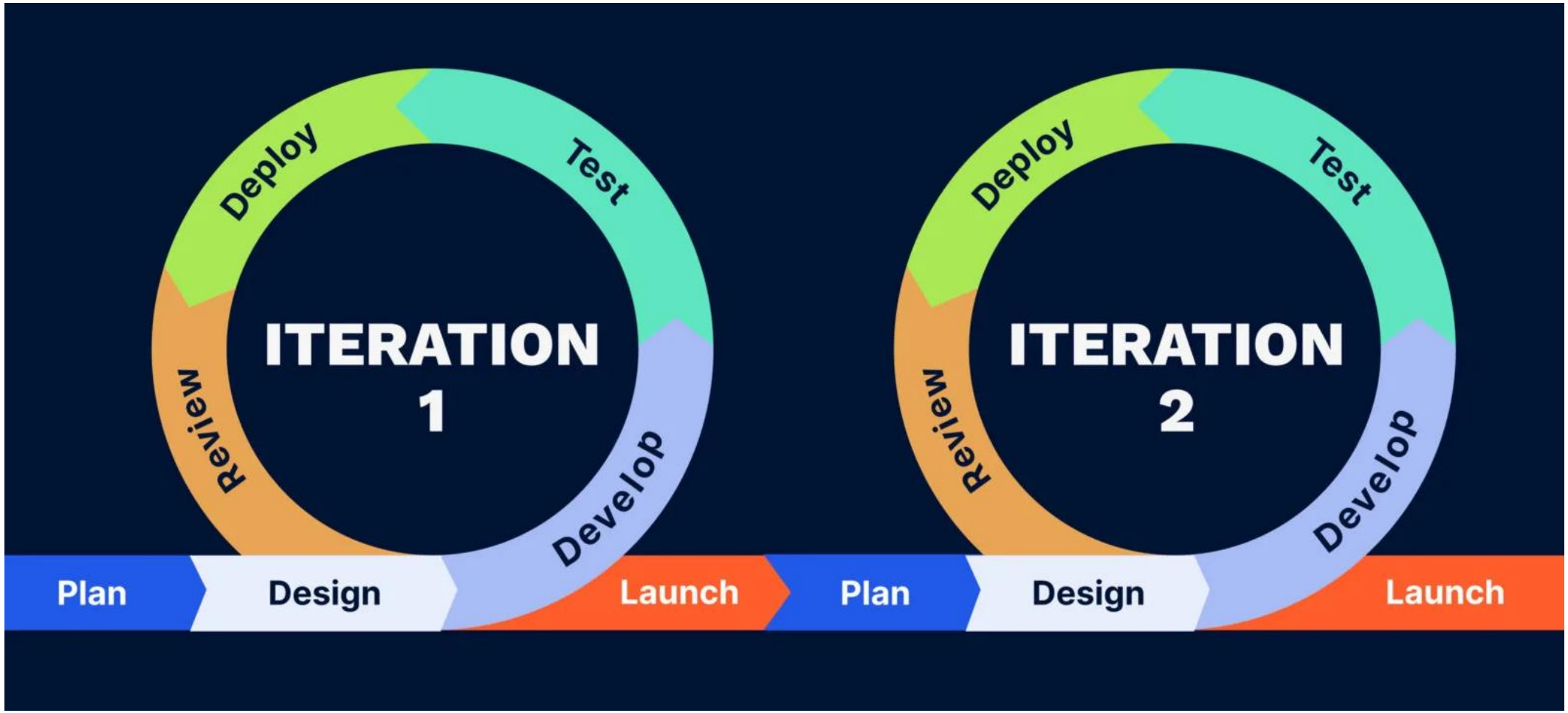
Unit II
Software Process Models and Project Management

Agile Software Development

- **Able to move quickly**
- Agile is the ability to create and respond to change.
- It is a way of **dealing with, and ultimately succeeding in, an uncertain and turbulent environment.**
- Agile is **a iterative approach to software delivery** that **builds software incrementally** from the start of the project, instead of trying to deliver all at once.
- The Agile model was primarily designed to help a project to **adapt to change requests quickly**. So, the main aim of the Agile model is to **facilitate quick project completion**.
- Agility is achieved by fitting the process to the project, removing activities that may not be essential for a specific project. Also, anything that is wastage of time and effort is avoided.

Agile Software Development



Agile Manifesto

- “We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:
  - Individuals and interactions over processes and tools
  - Working software over comprehensive documentation
  - Customer collaboration over contract negotiation
  - Responding to change over following a plan

  That is, while there is value in the items on the right, we value the items on the left more.”

Agile Principles

1. **Customer Involvement:** Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
2. **Incremental Delivery:** The software is developed in increments with the customer specifying the requirements to be included in each increment.
   - Frequent delivery of working software
3. **People not process:** The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes
   - Support, trust, and motivate the people involved
4. **Embrace Change:** Expect the system requirements to change and so design the system to accommodate these changes.
5. **Maintain simplicity:** Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system

Agile Principles

6. **Enable face-to-face interactions**
7. **Working software is the primary measure of progress**
8. **Agile processes to support a consistent development pace**
9. **Attention to technical detail and design enhances agility**
10. **Regular reflections on how to become more effective**
11. **Customer satisfaction through early and continuous software delivery**

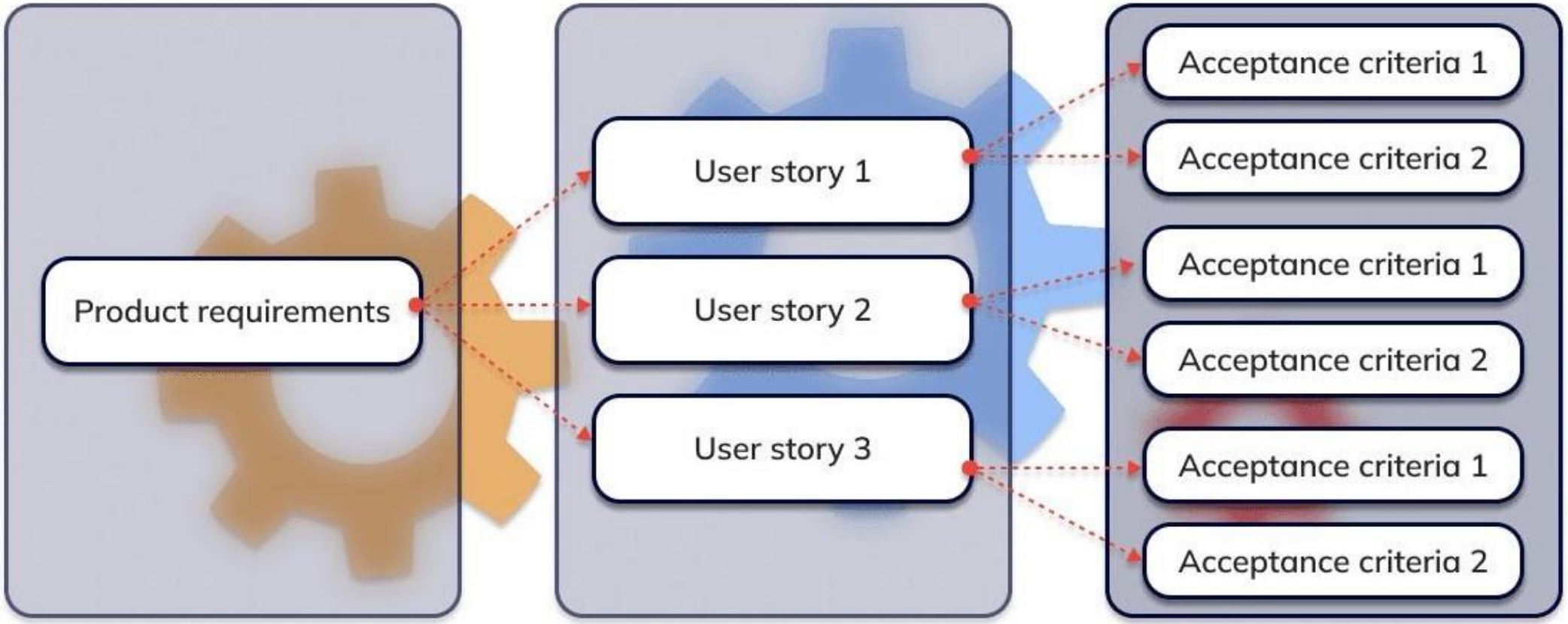
Agile Model -Benefits

1. Is a very realistic approach to software development.
2. Promotes teamwork and cross training.
3. Functionality can be developed rapidly and demonstrated.
4. Resource requirements are minimum.
5. Suitable for fixed or changing requirements
6. Delivers early partial working solutions.
7. Good model for environments that change steadily.
8. Minimal rules, documentation easily employed.
9. Enables concurrent development and delivery within an overall planned context.
10. Little or no planning required.
11. Easy to manage.
12. Gives flexibility to developers.

User Stories and Acceptance Criteria in Agile

- **User stories** provide a high-level understanding of a feature from the user's perspective, focusing on the what and why.
- **Acceptance criteria**, as mentioned, are detailed conditions that a feature must meet to be considered complete. They are more technical, offering a checklist that ensures the feature behaves as intended from an end-user perspective.
- Acceptance criteria establish a close connection to user stories, which capture requirements in a simple, concise format that is easy for everyone to understand
- Example: As a <*type of user*>, I want to <*perform some action*> so that <*some reason or benefit*>.
- As a traveler, I want to choose airplane seats online so that I have a window seat.

User Stories and Acceptance Criteria in Agile



Sprint and Sprint backlog

- A sprint is a short and fixed time frame during which a specific set of tasks are meant to be performed.
- An Agile project will be broken down into a certain number of sprints, each sprint lasting for a fixed duration of time.
- Usually, each sprint runs for 2–4 weeks.
- **A sprint backlog** is a list of work items the team plans to complete during a project sprint.
- These items are usually pulled from the product backlog during the sprint planning session.

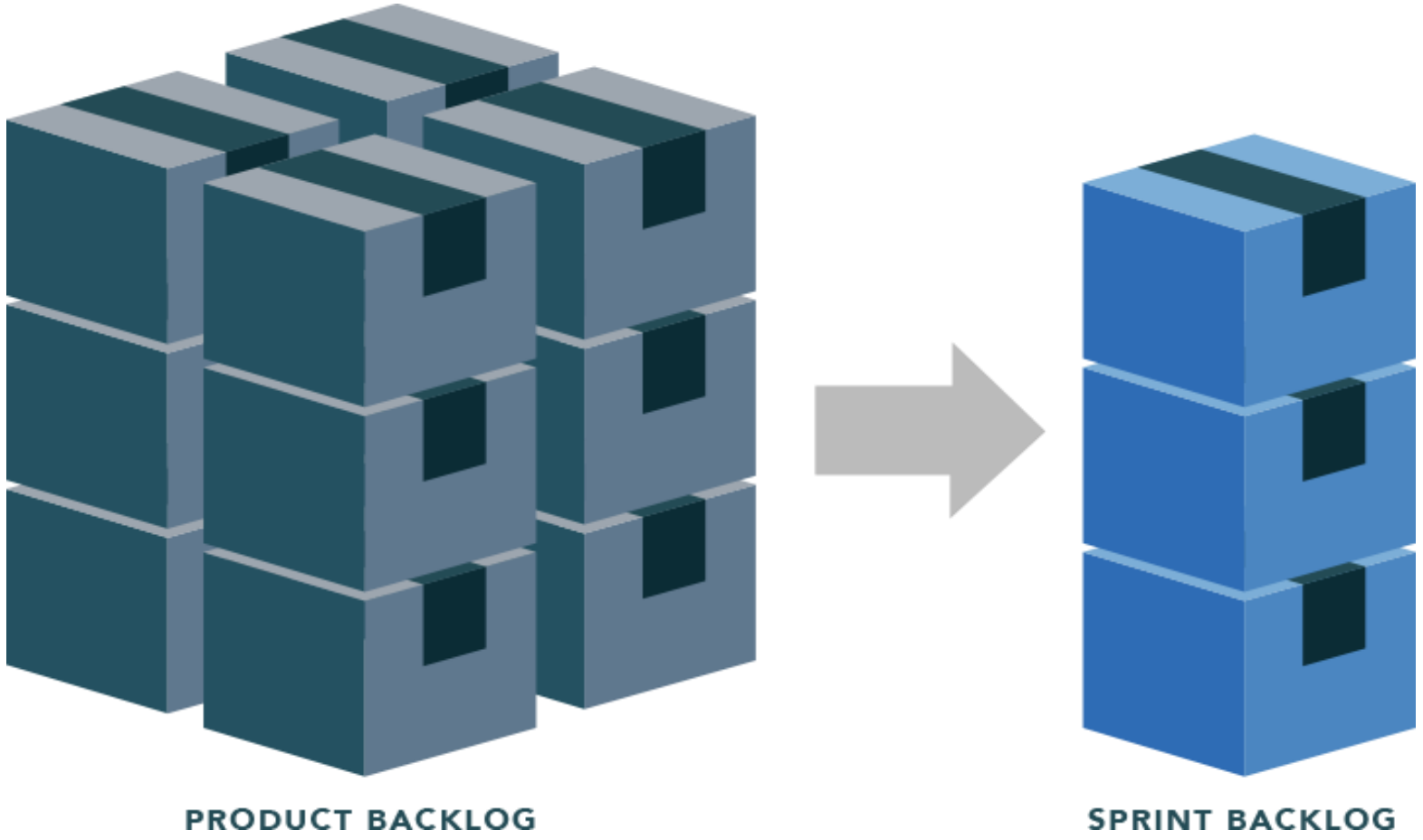
Product Backlog

- A product backlog is a **prioritized list of work for the development team** that is derived from the product roadmap and its requirements.
- A product backlog contains a prioritized list of **deliverables** (such as new features) that should be implemented as a part of project or product development in Agile development.
- Product backlogs are a decision-making **artifact** that help you estimate, refine, and prioritize everything you might sometime in the future want to complete.
- Typical items in a product backlog include **user stories**, **bug fixes**, **new features**, **changes to existing functionalities** and other related tasks.
- It helps ensure the team is working on the most important and valuable features, fixing the most important bugs, or doing other important work critical to product development.

Product Backlog

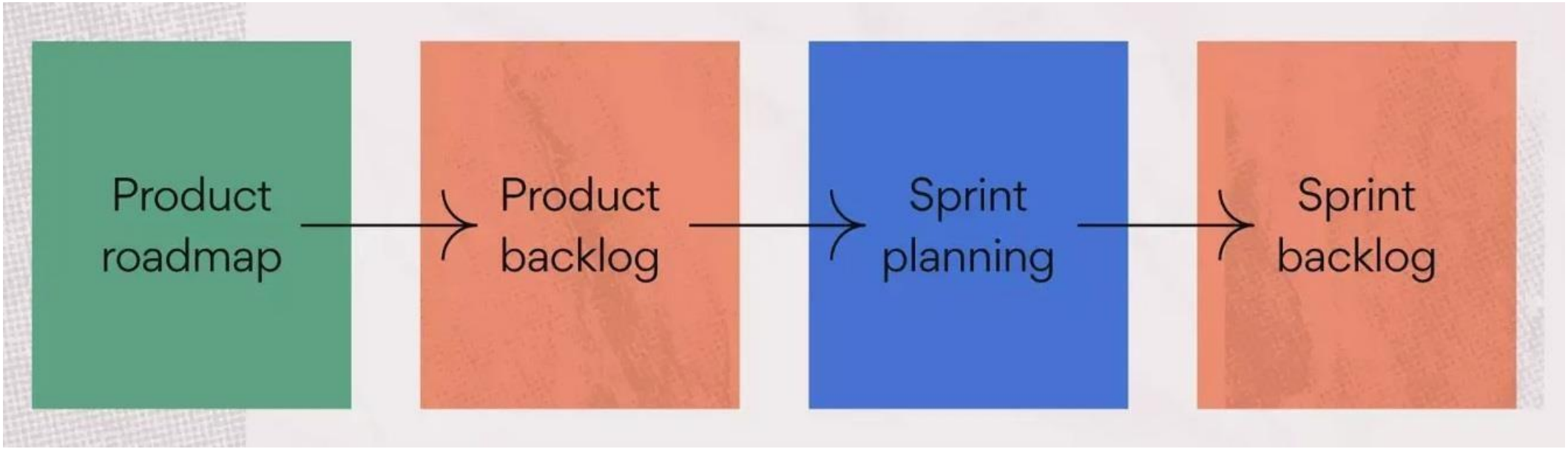
Item name	Status	Product backlog priority	Release tag
Idea Bucket Delegated to: All project members	**Not done**		
Social Media Game	**In progress**		
Theme 1	**In progress**		
Feature 1	**In progress**		
User story 1.1	Completed	Very high priority	Release 1 2018-02-19
User story 1.2	Completed	High priority	Release 1 2018-02-19
User story 1.3	In progress	Medium priority	Release 1 2018-02-19
Feature 2	**Not done**		
Theme 2 Visible to: Outsourced Vendor	**Not done**		

Product Backlog vs Sprint Backlog



A Sprint Backlog is a subset of Product Backlog

Product Backlog vs Sprint Backlog



Agile Scrum Methodology

- Scrum is a lightweight Agile project management framework that can be used to manage iterative and incremental projects of all types.
- The Scrum approach is a general agile method but its focus is on managing iterative development rather than specific agile practices.
- Each iteration consists of two- to four-week sprints, where each sprint's goal is to build the most important features first and come out with a potentially deliverable product.
- A sprint is a set period of time(generally 2-4 weeks) during which specific work has to be completed and made ready for review.
- More features are built into the product in subsequent sprints and are adjusted based on stakeholder and customer feedback between sprints.

Different roles in agile scrum methodology

- There are three core roles in agile scrum methodology:
    a. Scrum master
    b. Product owner
    c. Scrum team
- All of these people are committed to the scrum project.

a. Scrum Master

- The scrum master is the **facilitator of the scrum development process.**
- Scrum Master needs to understand Scrum well enough to train and mentor the other roles, and educate and assist other stakeholders who are involved in the process.
- The Scrum master is the leader of a Scrum team and is responsible for:

a. championing a project
b. providing guidance to the team and product owner
c. ensuring all agile practices are followed by team members.
d. holding daily meetings with the scrum team
e. Ensures that scrum rules are being enforced and applied as intended
f. coaching and motivating the team
g. ensuring that the team has the best possible conditions to meet its goals and produce deliverable products.

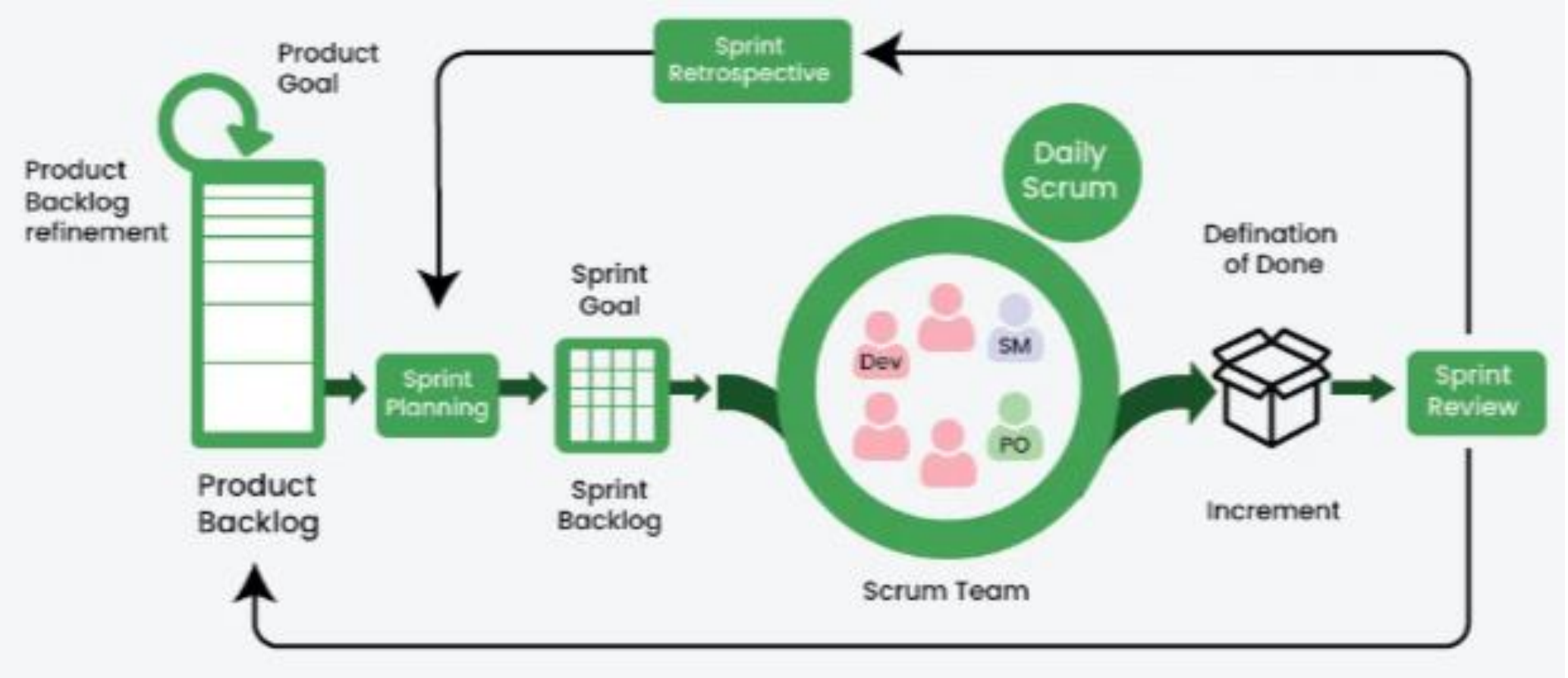
b. Product Owner

- The product owner **represents stakeholders**, which are typically customers.
- To ensure the scrum team is always delivering value to stakeholders and the business, the product owner determines:
  - product expectations
  - records **changes to the product and administers a scrum backlog**, a detailed and constantly updated to-do list for the scrum project.
- Product Owner **maintains the Product Backlog**, keeping it up to date and at the level of detail.
- The product owner is also responsible for prioritizing goals for each sprint, based on their value to stakeholders, such that the most important and deliverable features are built in each iteration.

c. Scrum Team

- The Team is a **self-organizing** and **cross-functional group of people** who do the hands-on work of **developing and testing the product**.
- The scrum team is a self-organized **group of three to nine individuals** who have the business, design, analytical and development skills to **carry out the actual work, solve problems and produce deliverable products.**
- Members of the scrum team self-administer tasks and are jointly responsible for meeting each sprint's goals.

Agile Scrum Methodology

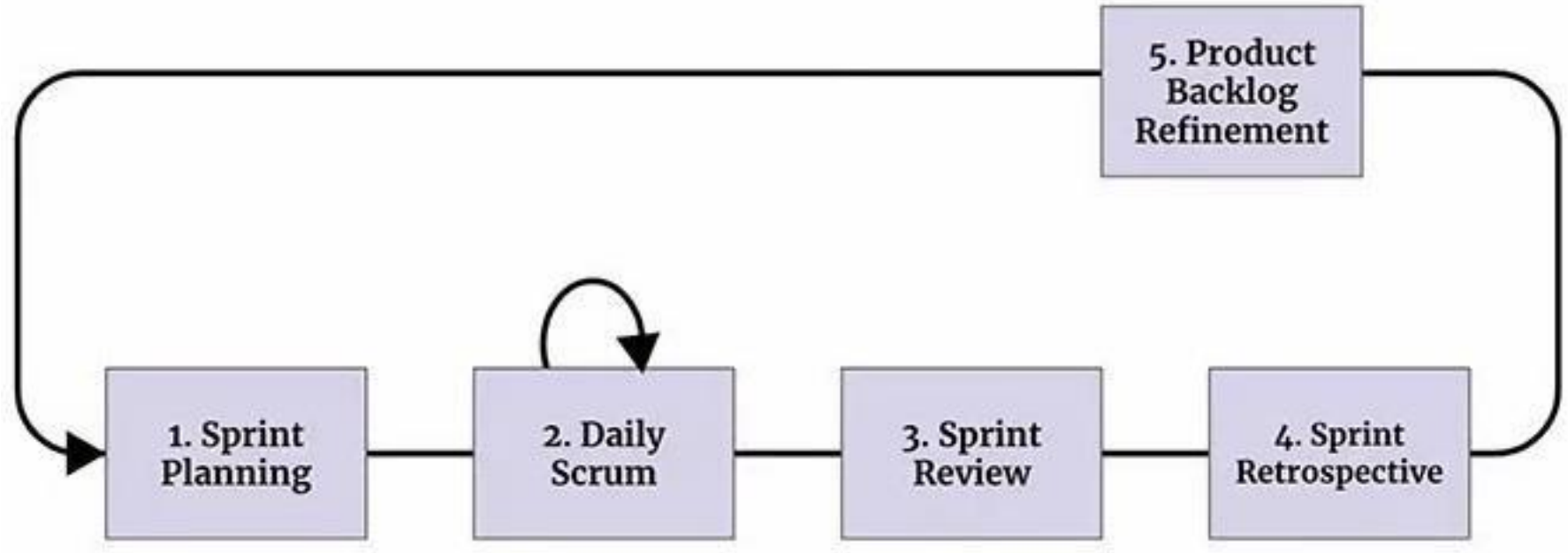


Agile Scrum Methodology

➢There are three phases in Scrum:

1. The initial phase is an outline planning phase where we establish the general objectives for the project and design the software architecture.
2. This is **followed by a series of sprint cycles**, where **each cycle develops an increment of the system.**
3. The project closure phase:
   - wraps up the project
   - completes required documentation such as system help frames and user manuals
   - assesses the lessons learned from the project.

Scrum Ceremonies



Scrum Ceremonies

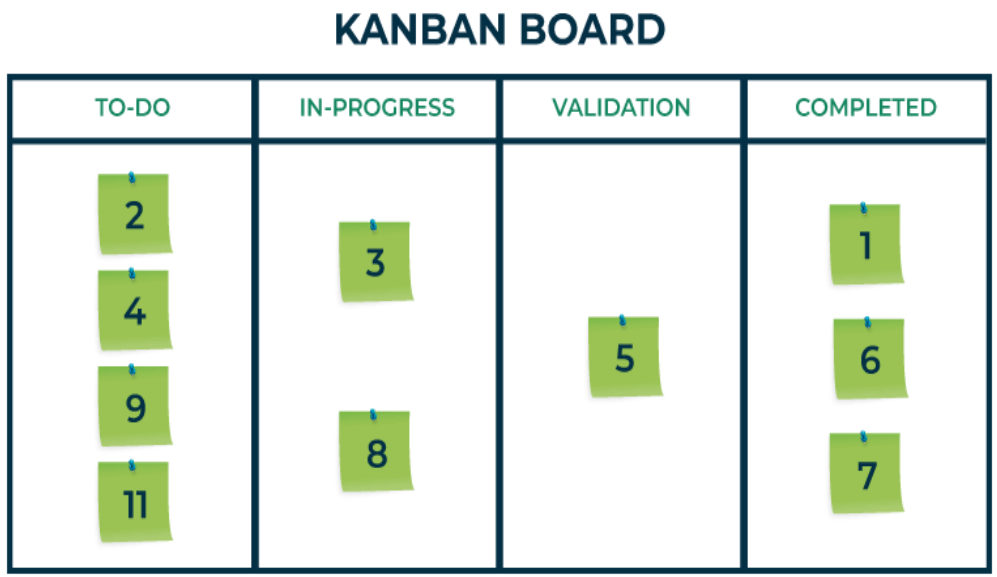
1. **Sprint Planning:** Sprint Planning initiates the sprint by laying out the work to be performed for the sprint. This resulting plan is created by the collaborative work of the entire Scrum team.
2. **Daily Scrum:** The purpose of the Daily Scrum is to inspect progress toward the Sprint Goal and adapt the Sprint Backlog as necessary, adjusting the upcoming planned work.
3. **Sprint Review:** Done at the end of a sprint. The purpose of the Sprint Review is to inspect the outcome of the sprint and determine future adaptations. The Scrum team presents the results of their work to key stakeholders, and progress toward the Product Goal is discussed.
4. **Sprint Retrospective:** The purpose of the Sprint Retrospective is to plan ways to increase quality and effectiveness. A sprint retrospective is a meeting in the Agile methodology for a team to reflect on its last sprint, **identifying what went well, what didn't, and how to improve future work**
5. **Product Backlog Refinement:** Involves refining of the backlog.

Kanban Methodology

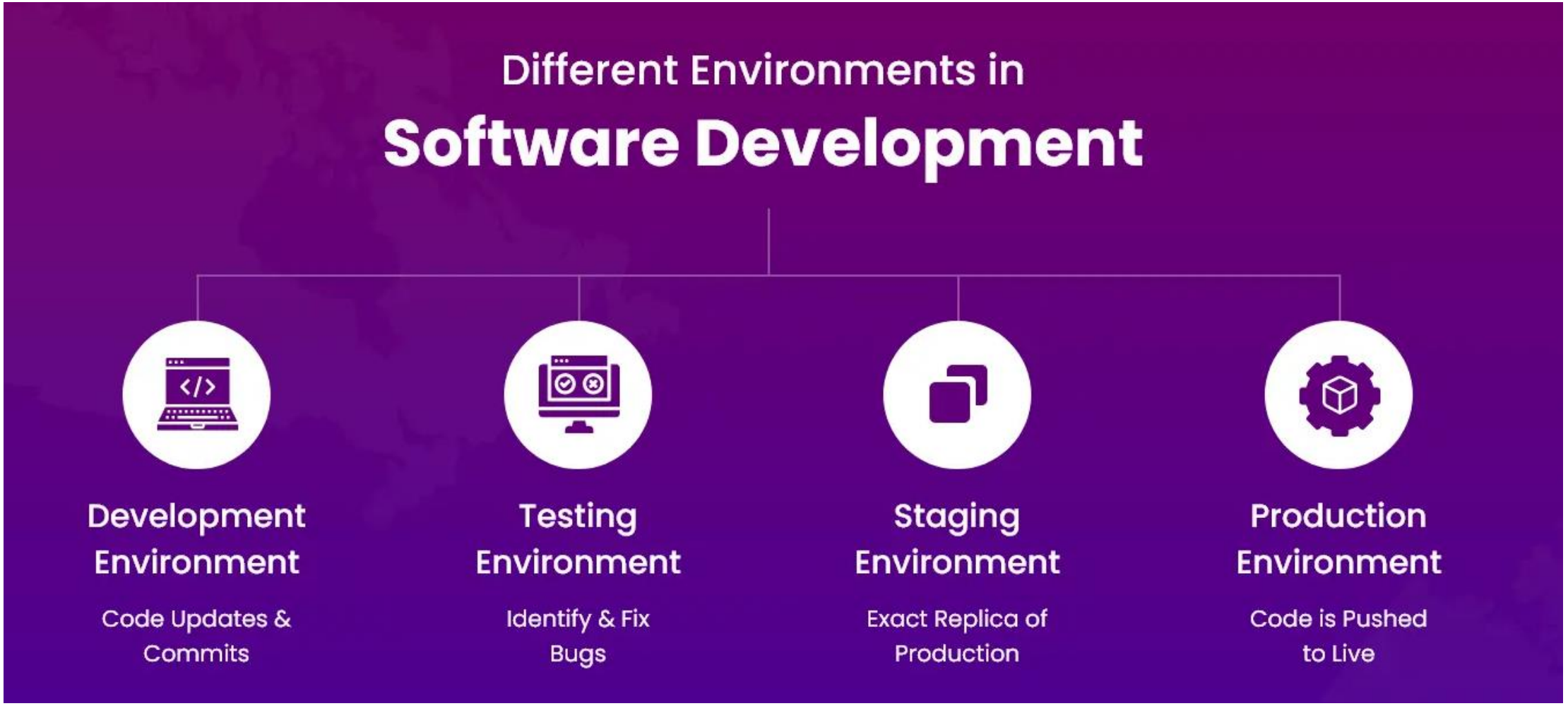
- Popular Agile Framework
- Kanban is a project management and workflow management methodology used to visualize work, limit work in progress, and improve efficiency.
- Kanban Methodology **visualizes work**, **limits Work-In-Progress(WIP)**, and **promotes continuous improvement** through transparent workflows.
- Teams use Kanban boards and cards to:
  - track tasks
  - identify bottlenecks
  - optimize delivery cycles.
- It mainly limits the Work In Progress (WIP). It restricts the number of tasks that can be worked on simultaneously.
- Kanban Methodoloy maintains a Kanban board which manages and organizes tasks in four different columns.

Kanban Methodology-Kanban Board

1. **To-do Column:** A To-Do list is formed in the first column of the board which **consists of all the user stories.** The individual cards must provide a brief description and necessary details of tasks like due date, functionalities, nature of task, etc.
2. **In-Progress Column:** In-Progress column consists of all the tasks that are currently under progress and under development phase. It acts as an indication that a team is currently operating working on it.
3. **Validation Column:** Code is reviewed, and the model is tested against several sets of tests like unit testing, acceptance testing, system testing, etc. All the functionalities are reviewed and verified.
4. **Done Column:** The done column reflects all the tasks that are finished and deployed without any errors.

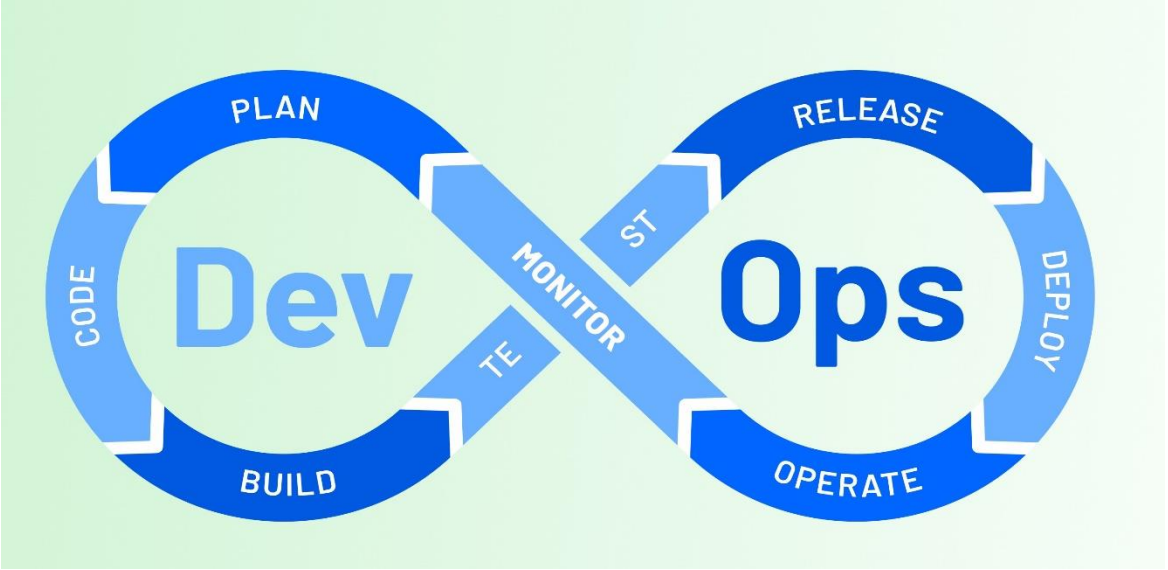


Software Development Environments



DevOps

- DevOps is a modern approach to software development that brings development and operations teams together to deliver applications faster and more reliably.
- A DevOps engineer is an IT professional who **bridges the gap between software development and IT operations.**
- They **optimize the software development lifecycle by automating repetitive tasks** like testing and infrastructure deployment **ensuring faster, safer, and more reliable software releases**



DevOps

Scenario

A developer commits code to a repository. A CI/CD pipeline automatically builds and tests the application. If all tests pass, the application is deployed to a server managed by the operations team. Monitoring tools detect any issues and provide feedback to developers for continuous improvement.

DevOps-Phases

1. **Plan**
   - **Key Activities:** Defining objectives, building plans, requirements gathering, analyzing and specifying, prioritizing tasks
   - **Tools:** Jira, Azure DevOps, Trello, etc.
2. **Code**
   - **Key Activities:** Writing code, version control, code reviews, branching and merging
   - **Tools:** Git, GitLab, Bitbucket, etc.
3. **Build**
   - **Key Activities:** Compile code, package applications, dependency management, artifact creation
   - **Tools:** Maven, Gradle, Apache Ant, etc.

DevOps-Phases

4. **Test**
   - **Activities:** Automated testing, unit testing, integration testing, security testing, performance testing
   - **Tools:** Junit, Selenim, Postman,etc
5. **Release**
   - **Activities:** The tested code is prepared, approved, and versioned so it is ready for deployment to staging or production environments. It mainly involves Change management approvals, artifact versioning, and release scheduling.
   - **Tools:** Jenkins, GitLab CI/CD, Azure DevOps, etc
6. **Deploy**
   - **Activities:** The application is automatically pushed to production servers or cloud environments so it becomes accessible to end-users. It involves Infrastructure provisioning, configuration, and automated environment deployment.
   - **Tools:** Kubernetes, Docker, Terraform

DevOps-Phases

7. **Operate**
   - **Activities:** The application is automatically pushed to production servers or cloud environments so it becomes accessible to end-users. It mainly involves Scaling resources, ongoing system maintenance, and patching.
   - **Tools:** Kubernetes, Puppet, Chef, etc
8. **Monitor**
   - **Activities:** The application is automatically pushed to production servers or cloud environments so it becomes accessible to end-users. It mainly involves Tracking vital metrics, system logging, and analyzing user behavior.
   - **Tools:** Grafana, DataDog, etc

DevOps-Role of DevOps Engineer

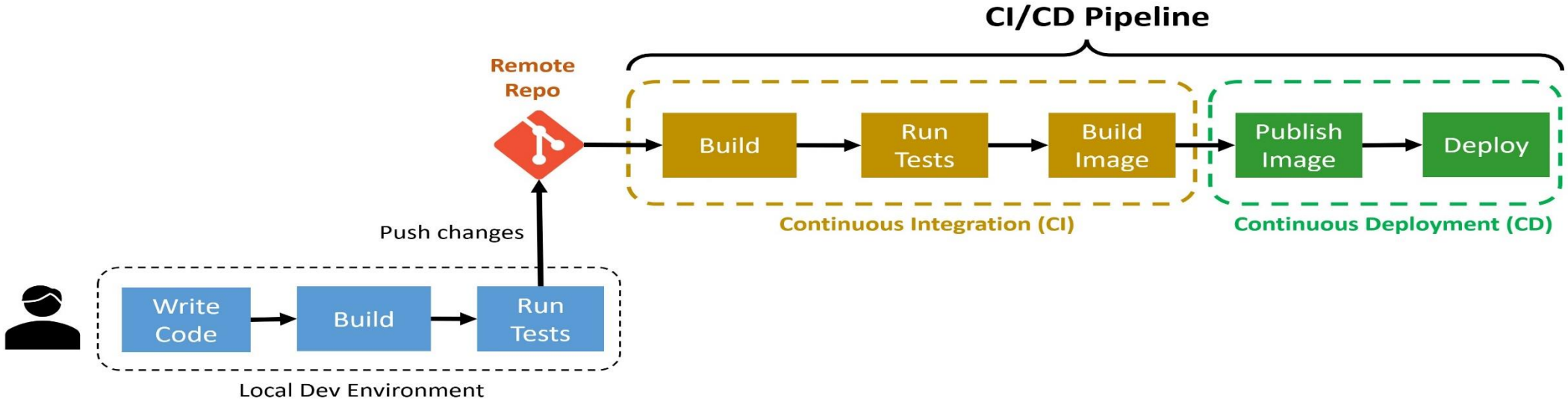
CI/CD Pipeline

- CI/CD stands for Continuous Integration/Continuous Deployment
- Backbone of DevOps
- CI →for auto-testing and merging
- CD →for releasing reliable software updates securely and quickly to end-users\
- **Continuous Integration (CI)**: A software engineering practice where developers regularly merge their code changes into a central repository, after which automated builds and tests are run.
- **Continuous Deployment (CD)**: A software engineering practice in which code changes are delivered frequently through automated deployments.

DevOps-Role of DevOps Engineer

CI/CD Pipeline

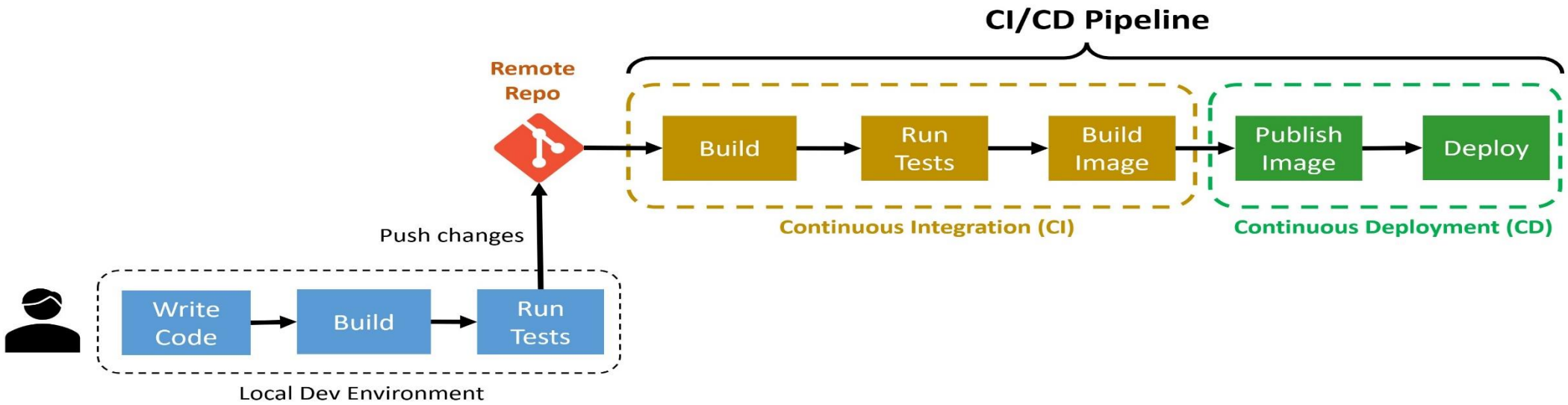
- CI transforms raw source code into a functional, production-ready application through a series of rigorous stages. If any stage fails, the pipeline halts preventing faulty code from ever reaching the end user.
- CI involves automatically building and testing code changes as they are committed to the repository. This stage ensures that the changes made by developers do not break the existing codebase.



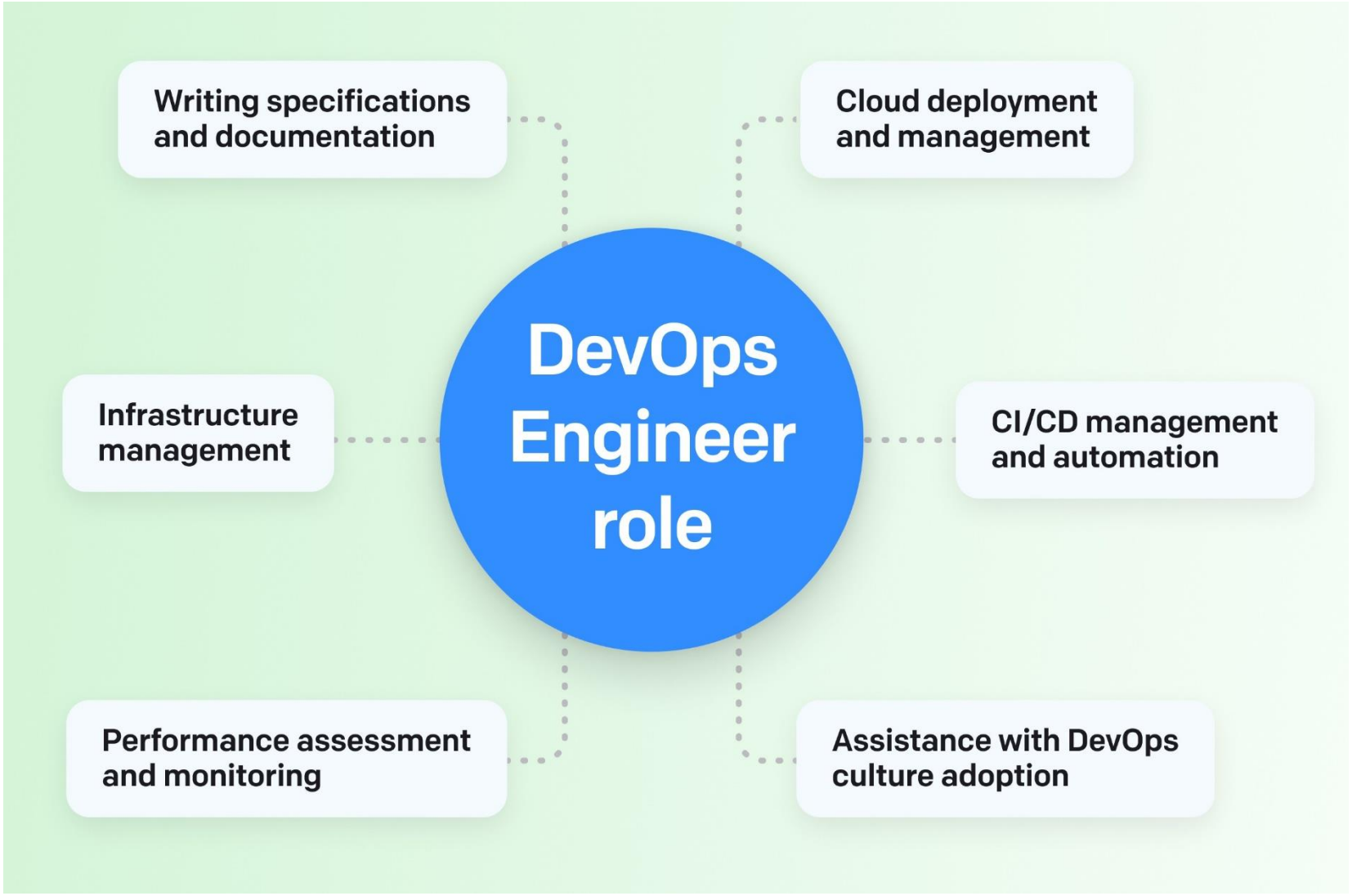
DevOps-Role of DevOps Engineer

CI/CD Pipeline

- Continuous delivery (CD) is the process of automatically preparing tested code so it is always ready for deployment to any environment.
- Once code has been tested and built as part of the CI process, CD takes over during the final stages to ensure it's packaged with everything it needs to deploy to any environment at any time.



DevOps-Role of DevOps Engineer



DevOps-Collaboration between Development and Operations

- Development and operations staff must manage constantly changing combinations of applications and target environments.
- To manage and automate deployment, development and operations staff collaborate on a number of assets, including the following:
  a. Software Requirements
  b. Deployment Environment
  c. Deployment Plan
  d. Automation Plan

Why Collaboration Matters?

- **Shared Ownership:** Both teams align around the same objectives ensuring high-quality code is safely deployed and continuously available.
- **Faster Problem Resolution:** By integrating early in the lifecycle, teams eradicate the "it works on my machine" friction.
- **Increased Transparency:** Eliminating communication gaps creates better visibility into system performance and infrastructure needs.

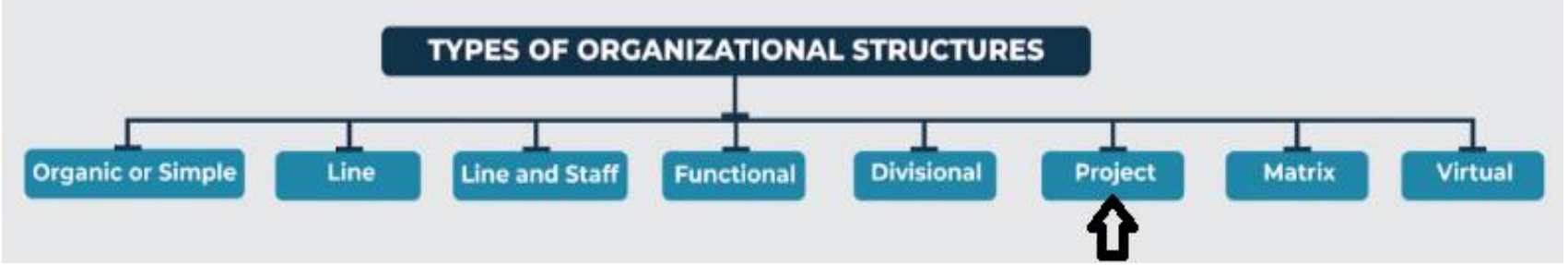
DevOps-Collaboration between Development and Operations

Core Practices Driving Collaboration

1. CI/CD
2. Infrastructure as Code (IaC)
   - DevOps practice where **Infrastructures such as servers, networks, and cloud resources are automatically provisioned and managed using code** instead of manual configuration.
   - With IaC, **configuration files are created** that contain our infrastructure specifications, which makes it easier to edit and distribute configurations.
   - It helps maintain consistency, speed, and reliability across deployments.
3. Unified Tooling: Using **shared monitoring, logging, and project tracking platforms** so developers and system administrators have a single source of truth.
4. Proactive Feedback Loops: Proactive feedback loops in DevOps are **automated mechanisms embedded throughout the Software Development Life Cycle (SDLC)** designed to **anticipate, catch, and resolve issues** before they impact end-users or reach production. They accelerate the delivery of high-quality software while vastly improving system reliability.

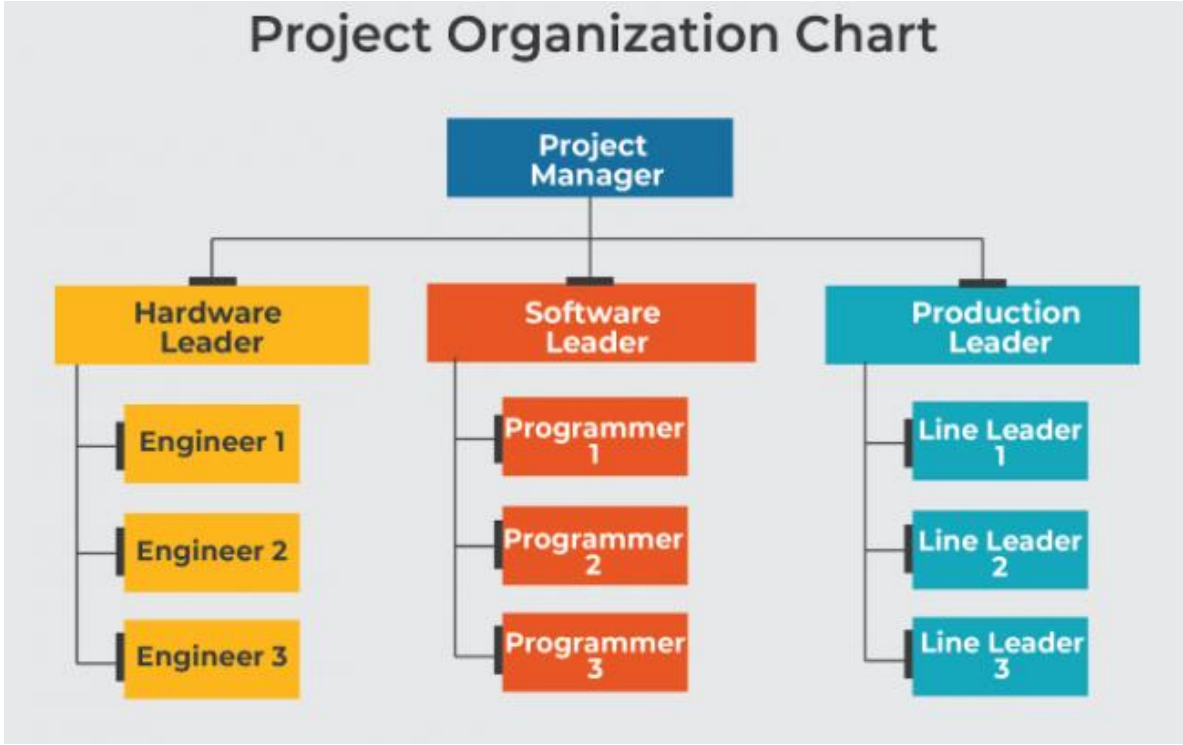
Common Organizational Structures in Project Management

- An organizational structure is a standard hierarchy of operations
- It defines how we can divide, coordinate and direct groups.
- It defines the positions and describes the tasks required to achieve an organization's objectives and vision.



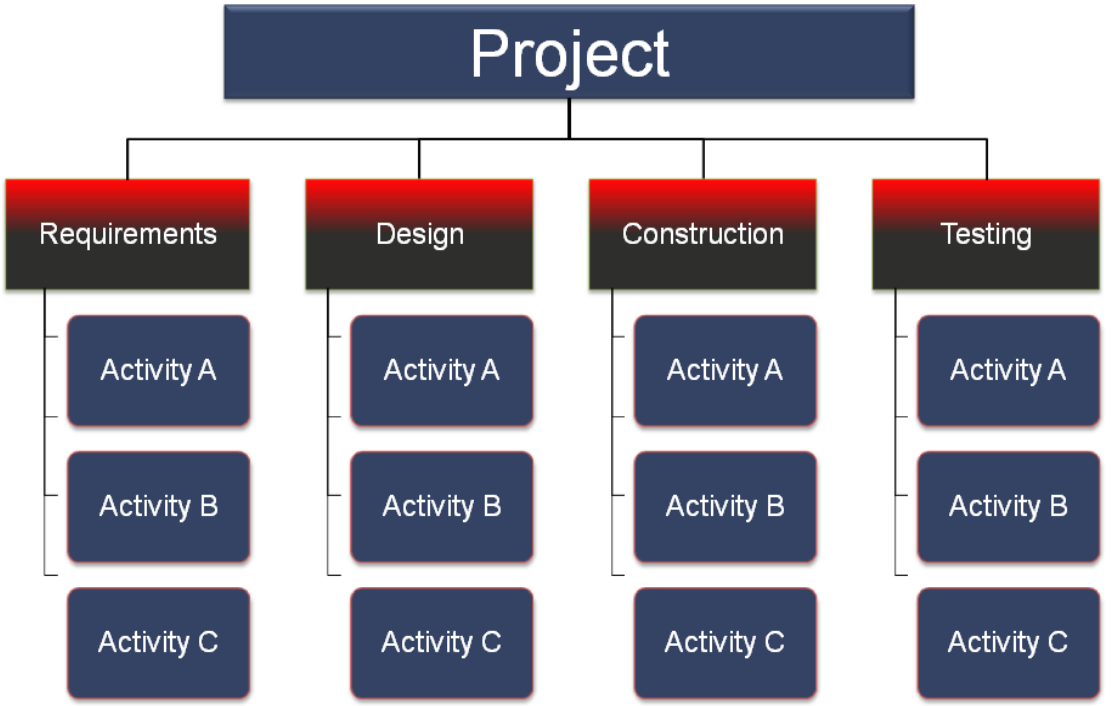
Project Organization and Roles

- Project organization refers to the style of coordination, communication and management a team uses throughout a project's life cycle in order to achieve organization's goals and objectives.
- It specifies how teams are structured and managed to execute software projects.
- It provides a systematic roadmap for managing the project by clearly defining participants, their responsibilities, reporting relationships, and communication channels.
- Its primary purpose is to ensure accountability, streamline communication, and align resources to deliver project objectives efficiently.



WBS(Work Breakdown Structure)

- A work breakdown structure (WBS), in project management and systems engineering, is a deliverable-oriented decomposition of a project into smaller components.
- Dividing complex projects to simpler and manageable tasks is the process identified as Work Breakdown Structure (WBS).
- Usually, the project managers use this method for simplifying the project execution.
- In WBS, much larger tasks are broken down to manageable chunks of work. These chunks can be easily supervised and estimated.



Task Assignment

- Task Assignment is the process of delegating specific documentation or project responsibilities to team members, often with deadlines and tracking capabilities.
- It is the strategic process of distributing specific project duties to team members, ensuring accountability, and optimal resource utilization.
- It relies on:
  a. **Breaking of Task:** breaking complex projects into smaller units **(Work Breakdown Structure)**
  b. **Mapping tasks:** mapping tasks to individual skills
  c. **Tracking work products:** tracking the resulting work products (deliverables)
- Effective assignment of work requires alignment between the task demands and the team's capabilities.

Work Products

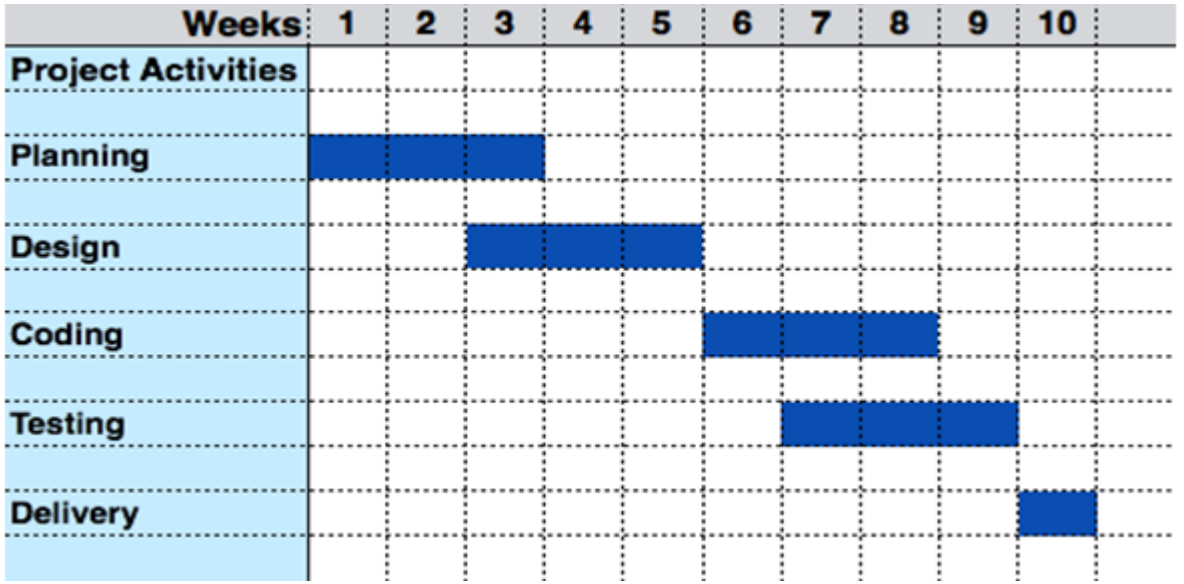
- In terms of software engineering, work products are any artifacts, documents, or deliverables created throughout the software development life cycle (SDLC).
- Work products can be categorized as:
  a. **Tangible Work Product:** These are physical or digital items we can view, store, and transfer, such as source code and documentation. Examples include Requirements Documents, Design blueprints, executable software, testing and quality reports, user manuals, APIs, etc.
  b. **Intangible Work Product:** These are the conceptual, procedural, and intellectual assets created during the software development lifecycle. They lack physical form. They generally include intellectual properties such as patients, copyrights, trademarks, etc.

Representing and Scheduling Project Plans

- A project manager has a wide variety of techniques available for depicting (to represent or show something in a picture or story) and documenting project plans.
- These planning documents can take the form of graphical or textual reports, although graphical reports have become most popular for depicting project plans.
- The most commonly used methods are **Gantt charts** and **network diagrams**.
- Gantt charts **do not (typically) show how tasks must be ordered (precedence)** but simply show **when a task should begin and when it should end**, they are often more useful for depicting (representing) relatively simple projects.
- A network diagram shows the **ordering of activities by connecting a task to its predecessor and successor tasks**.
- Sometimes a network diagram is preferable; other times a Gantt chart more easily shows certain aspects of a project.

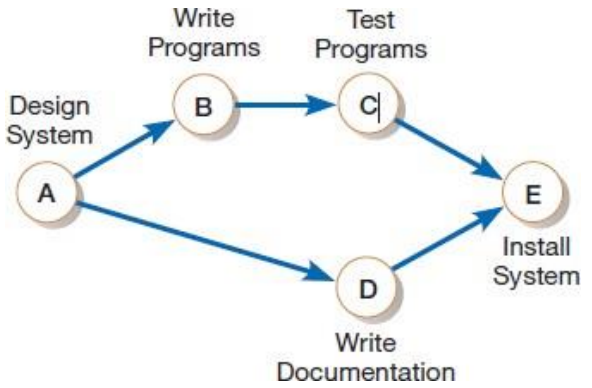
Gantt Charts

- Gantt chart is a type of a bar chart that is used to allocate resources to activities including staff, hardware and software.
- Gantt charts allow project managers to track the progress of the entire project. Through Gantt charts, the project manager can keep a track of the individual tasks as well as of the overall project progression.
- The bars in the Gantt are drawn along a time line and the length of each bar is proportional to the duration of time planned for the corresponding activity.



Network Diagrams

- A network diagram is a visual representation of the workflow of a project.
- A network diagram is a chart that is populated with boxes noting tasks and responsibilities, and then arrows that map the schedule and the sequence that the work must be completed.
- Network diagramming is a **critical path scheduling** technique used for controlling resources.
- **A critical path refers to a sequence of task activities whose order and durations directly affect the completion date of a project**.
- Network Diagrams are mainly used for:
  - Planning the structure of a project
  - Coordinating updates or changes automatically
  - Identifying potential risks or bottlenecks of a project
  - Documentation for external communication
  - Keeping track of project progress



CPM(Critical Path Methods)

- CPM allows us to monitor the achievements of the project goals and help to find the remedial actions need to be taken to get the project back on course.
- Critical path is the sequential activities from start to the end of a project i.e. the path of longest duration as determined on the project network diagram(total duration of the project).
- In the critical path method, the critical activities of a program or a project are identified. These are the activities that have a direct impact on the completion date of the project.
- Although many projects have only one critical path, some projects may have more than one critical paths depending on the flow logic used in the project.
- If there is a delay in any of the activities under the critical path, there will be a delay of the project deliverables.
- Most of the times, if such delay is occurred, project acceleration or re-sequencing is done in order to achieve the deadlines.

CPM(Critical Path Methods)

- Critical path method is based on mathematical calculations and it is used for scheduling project activities. This method was first introduced in 1950s as a joint venture between Remington Rand Corporation and DuPont Corporation.
- The critical task will have starting time and finishing time and **task not in critical path can have flexibility of starting and finishing the task known as slack time or float time.**
- Critical Path have four key elements:
  a. **Critical path analysis:** find the path with longest duration among different activities from start to finish.
  b. **Analyze the Earliest Start Time(EST) and Latest Finish Time (LFT)**
     EST→how fast/early an activity can start
     LFT→ Maximum time available within which the project must be finished
  c. **Calculate Earliest Finish Time(EFT) and Latest Start Time(LST) where**
     EFT = EST + activity duration
     LST = LFT – activity duration

CPM(Critical Path Methods)

d. **Float (slack) determination:** float is an amount of time an activity can be delayed before it causes whole project delayed that is slack.(no slack in CP)

   Total Float = LST – EST or LFT – EFT

   The path with total float =0 is the critical.

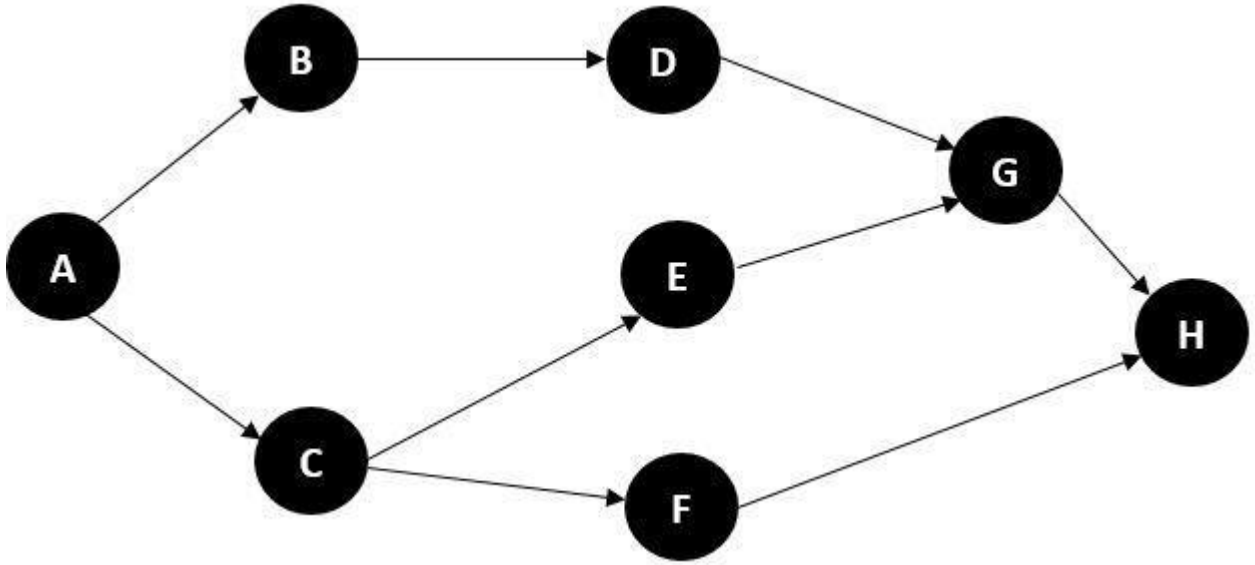
CPM Numerical

Find the project duration and critical path for following dependencies using CPM.

Activity	Predecessor	Duration (days)
A	-	3
B	A	4
C	A	2
D	B	5
E	C	1
F	C	2
G	D,E	4
H	F,G	3

CPM Numerical

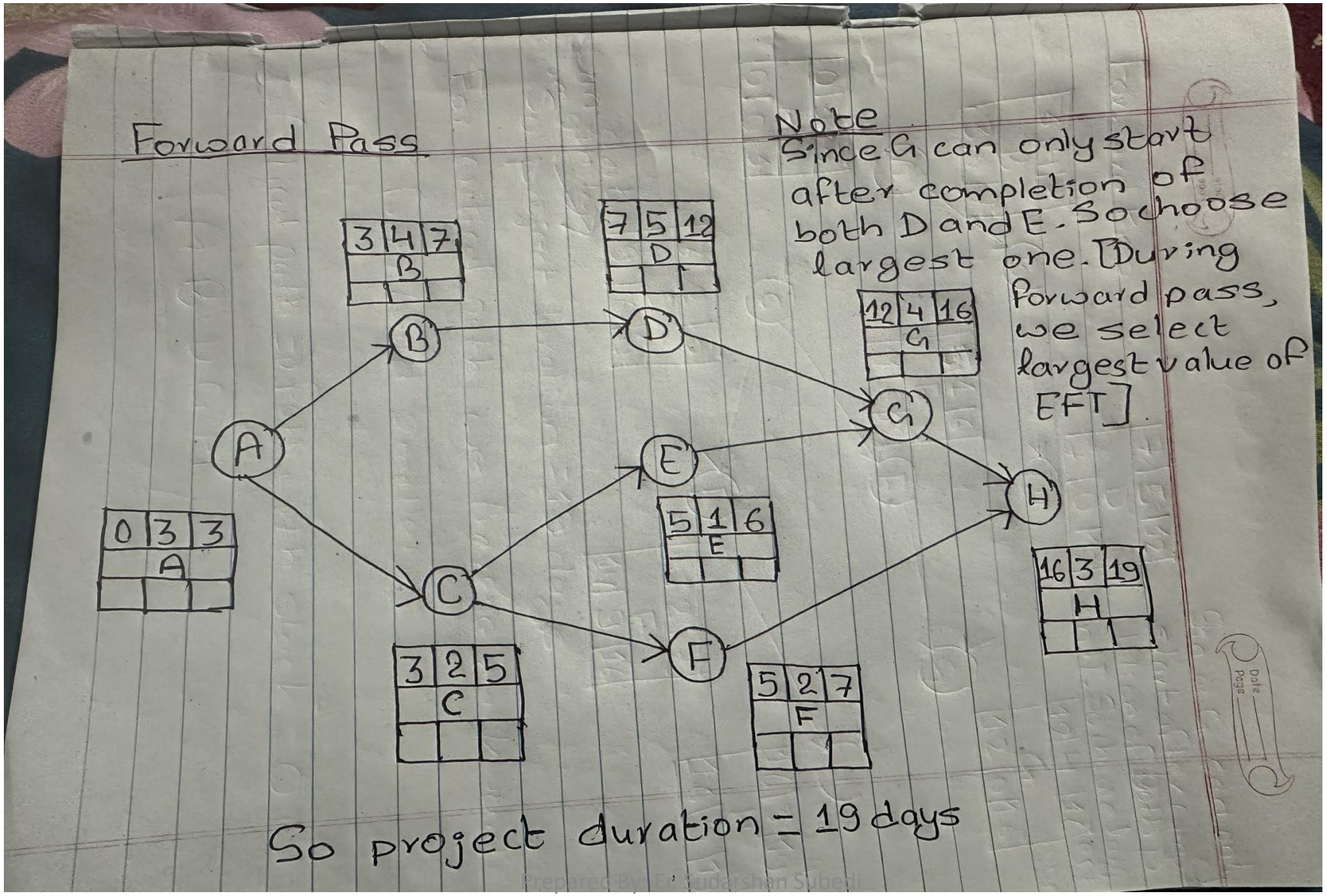
First we create a network diagram as below:



Now for each activity, we compute:

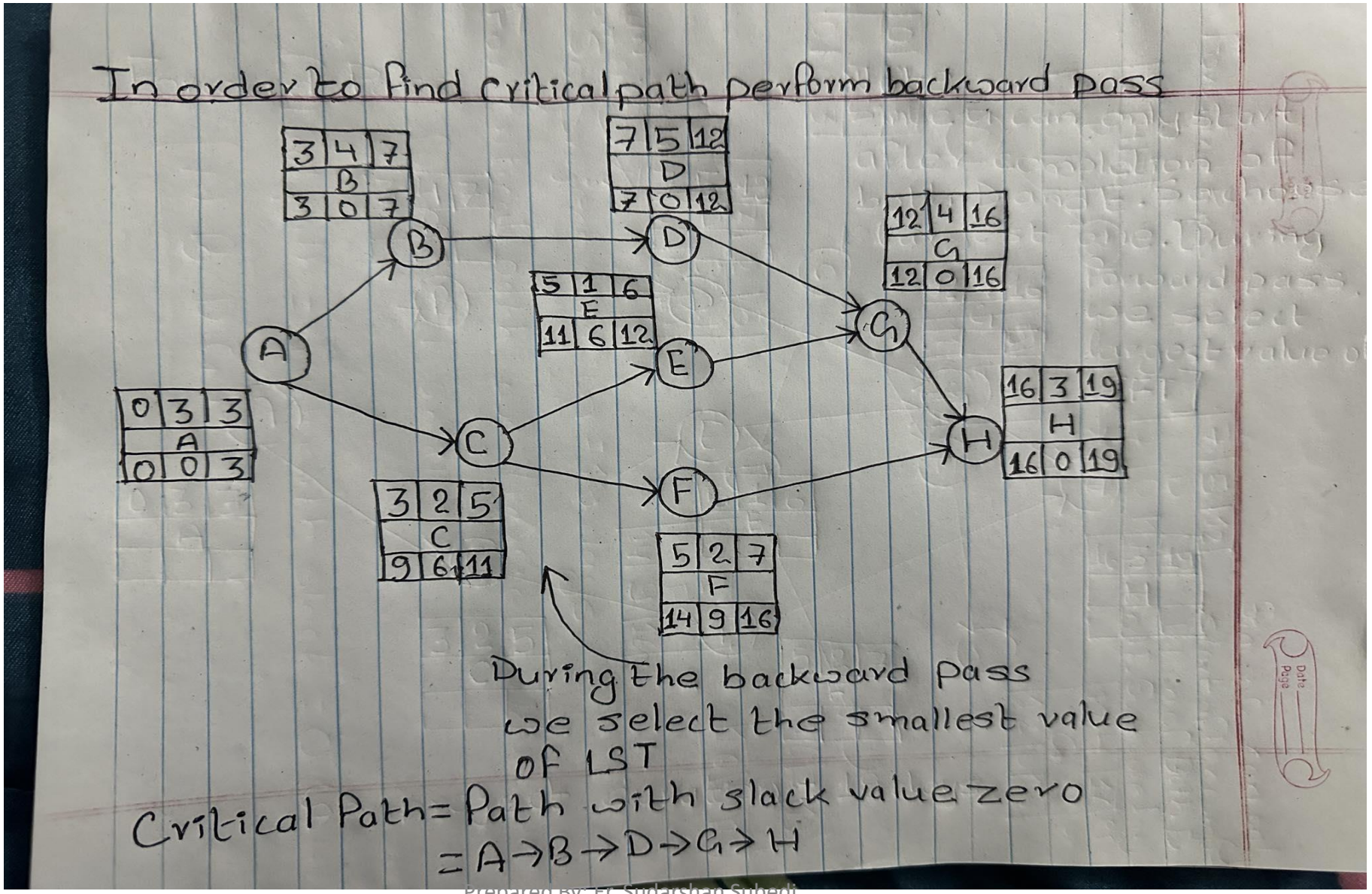
EST	Duration	EFT
Activity/Node Name		
LST	Slack	LFT

CPM Numerical



Prepared By: Er. Sudarshan Subedi

CPM Numerical



CPM Numerical

Find the project duration and critical path for following dependencies using CPM.

Activity	Predecessor	Duration (days)
A	-	5
B	A	4
C	A	5
D	B	6
E	C	3
F	D,E	4

PERT(Program Evaluation and Review Techniques)

- PERT (Program Evaluation and Review Technique) is a project management tools used to schedule, organize and coordinate tasks within a project.
- PERT was initially created by the US Navy in the late 1950s. The pilot project was for developing Ballistic Missiles and there have been thousands of contractors involved.
- PERT **chart represents the statistical variations in the project estimates assuming normal distribution.** So instead of making single estimates for each task ,pessimistic(p),optimistic(o) and most likelihood(m) estimates are made. That is
  - Expected Time Estimate(te)=$\frac{(to+4*tm+tp)}{6}$
  - Where optimistic time(to) means the minimum possible period of time for an activity to be completed
  - Pessimistic time(tp) means the maximum possible period of time for an activity to be completed
  - Most likelihood or realistic time(tm) is the project manager's best guess of the amount of time that the activity actuallyw ill require for the completion

PERT(Program Evaluation and Review Techniques)

- In order to find the variance of each activity in the Network Diagram we can use the formula

$$\sigma^2 = \left(\frac{tp - to}{6}\right)^2$$

- If we want to calculate the value of standard normal variable, we can calculate using following formula:

$$Z = \frac{Ts - Te}{\sigma}$$

PERT Numerical

Consider the following data.

Activity	Predecessor	Duration in weeks		
		to	tm	tp
A	-	5	6	7
B	-	1	3	5
C	-	1	4	7
D	A	1	2	3
E	B	1	2	9
F	C	1	5	9
G	C	2	2	8
H	E,F	4	4	10
I	D	4	4	10
J	I,H,G	2	2	8

1. Draw the network diagram
2. Find the expected duration of project/ project length.
3. Find the critical path.
4. Find the probability of project completing in 30 days.

PERT Numerical

Steps to solve:

1. For each activity, compute the expected time estimate as:

$$\text{Expected Time Estimate(te)} = \frac{(to + 4 * tm + tp)}{6}$$

2. Now draw the circuit diagram as per te.
3. Find the project duration and critical path as in CPM.
4. In order to find the probability, find the z score and observe the a-score table.

Project Communication

- Communication plays a vital role in success of a project
- It is vital for smooth information flow among involved parties.
- It is the structured process of exchanging information across development teams, clients, and stakeholders.



Project Communication-Planned and Unplanned Communication

Aspect	Planned Communication	Unplanned Communication
Definition	Communication that is scheduled, organized, and follows a predefined process.	Communication that occurs spontaneously without prior scheduling.
Purpose	To share formal information, track progress, and make decisions.	To quickly exchange information, solve immediate problems, or clarify issues.
Documentation	Usually documented and recorded.	Often undocumented and informal.
Structure	Structured and follows an agenda.	Flexible and informal.
Frequency	Occurs at predetermined times.	Occurs whenever needed.

Project Communication-Communication Mechanism

- Communication mechanisms are the methods or channels through which information is exchanged among project stakeholders, developers, managers, clients, and users.
- Communication Mechanisms:

1. **Formal Communication:** Official and structured communication that follows organizational procedures. Examples: Project meetings, status reports, documentations, presentations, etc.
2. **Informal Communication:** Casual and spontaneous communication among team members. Examples: Quick chats, Phone calls, Instant messages, etc.
3. **Verbal Communication:** Information is exchanged through spoken words. Examples: Daily stand-up meetings, telephone calls, video conferences, etc.
4. **Written Communication:** Information is communicated in written form. Examples: Emails, reports, plans, user manuals, etc.

Project Communication-Communication Mechanism

5. **Electronic Communication:** Communication using digital technologies. Examples: Email, Chat Applications, Video Conferencing, etc

Project Communication-Communication Tools

1. **Email Tools:** Gmail, Microsoft Outlook, etc
2. **Messaging and Team Chat Tools:** Slack, Microsoft Teams, Discord, etc
3. **Video Conferencing Tools:** Zoom, Google Meet, Microsoft Tools, etc.
4. **Project Management Tools:** Jira, Asana, Trello, etc.
5. **Version Control and Collaboration Tools:** Github, GitLab, Bitbucket, etc.

Software Reviews

- **Software Review** is a **systematic inspection of software** by one or more individuals who work together **to find and resolve errors and defects in the software during the early stages** of the Software Development Life Cycle (SDLC).
- A software review is an essential part of the Software Development Life Cycle (SDLC) that helps software engineers in **validating the quality, functionality, and other vital features and components of the software.**
- It is a whole process that includes testing the software product and it makes sure that it meets the requirements stated by the client.

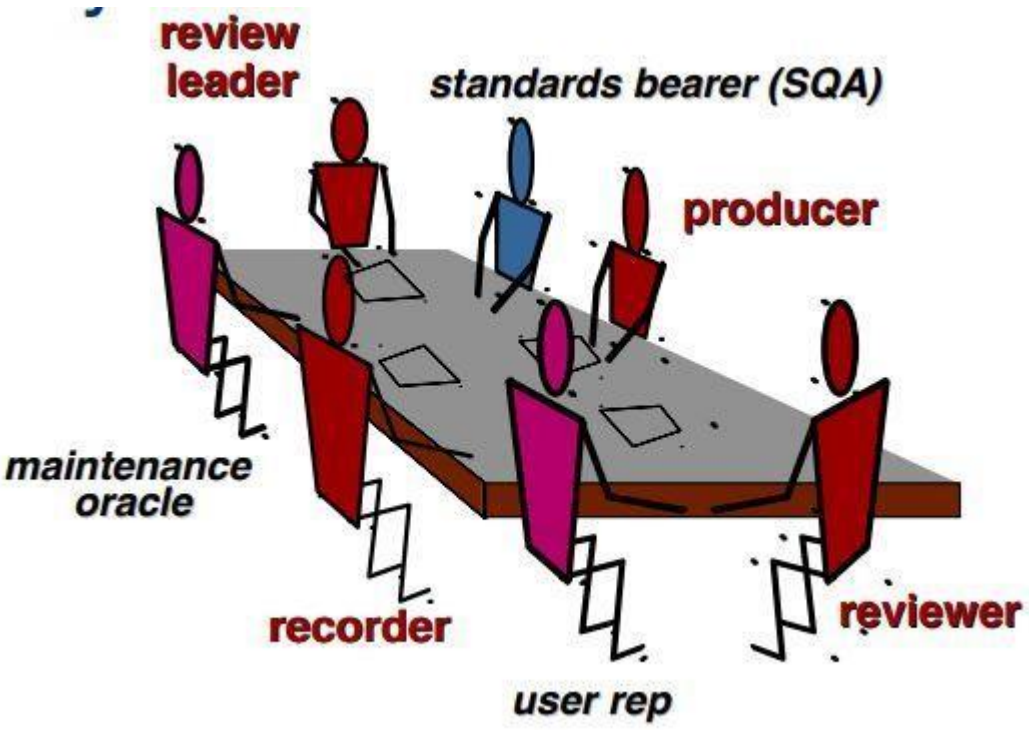
Formal Technical Review (FTR)

- Formal Technical Review (FTR) is a software quality control activity performed by software engineers.
- It is an organized, methodical procedure for assessing and raising the standard of any technical paper, including software objects.
- **Finding flaws**, **making sure standards are followed**, and **improving the product or document under review's overall quality** are the main objectives of a formal technical review (FTR).
- Although FTRs are frequently utilized in software development, other technical fields can also employ the concept.

Formal Technical Review (FTR)

- Objectives
  - To uncover errors in function, logic, or implementation for any representation of the software
  - To verify that the software under review meets its requirements
  - To ensure that the software has been represented according to predefined standards
  - To achieve software that is developed in a uniform manner
  - To make projects more manageable
- Serves as a training ground for junior software engineers to observe different approaches to software analysis, design, and construction
- Promotes backup and continuity because a number of people become familiar with other parts of the software

Formal Technical Review (FTR)-The Players



1. **Producer**—the individual who has developed the work product. Producer informs the project leader that the work product is complete and that a review is required
2. **Review leader**—evaluates the product for readiness, generates copies of product materials, and distributes them to two or three reviewers for advance preparation.
3. **Reviewer(s)**—expected to spend between one and two hours reviewing the product, making notes, and otherwise becoming familiar with the work.
4. **Recorder**—reviewer who records (in writing) all important issues raised during the review

The FTR Meeting

- 3-5 people are involved in the meeting
- FTR focuses on a specific work product (a software requirements specification, a detailed design, a source code listing)
- **Activities before the meeting**
  - The producer informs the project manager that a work product is complete and ready for review
  - The project manager contacts a review leader, who evaluates the product for readiness, generates copies of product materials, and distributes them to the reviewers for advance preparation
  - Each reviewer spends one to two hours reviewing the product and making notes before the actual review meeting
  - The review leader establishes an agenda for the review meeting and schedules the time and location

The FTR Meeting (continued)

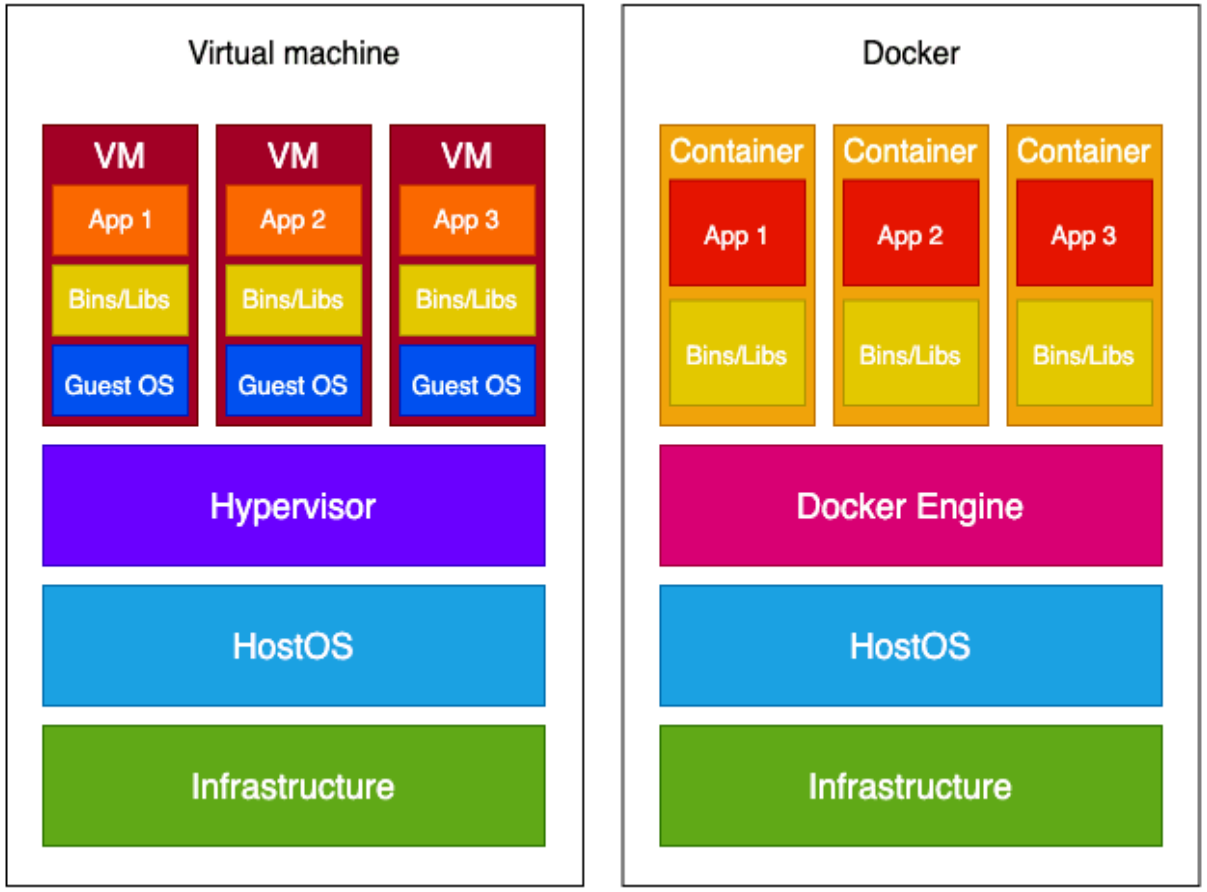
- **Activities during the meeting**
  - The meeting is attended by the review leader, all reviewers, and the producer
  - One of the reviewers also serves as the recorder for all issues and decisions concerning the product
  - After a brief introduction by the review leader, the producer proceeds to "walk through" the work product while reviewers ask questions and raise issues
  - The recorder notes any valid problems or errors that are discovered; no time or effort is spent in this meeting to solve any of these problems or errors
- **Activities at the conclusion of the meeting**
  - All attendees must decide whether to
    - Accept the product without further modification
    - Reject the product due to severe errors (After these errors are corrected, another review will then occur)
    - Accept the product provisionally (Minor errors need to be corrected but no additional review is required)
    - All attendees then complete a sign-off in which they indicate that they took part in the review and that they concur with the findings

FTR Guidelines

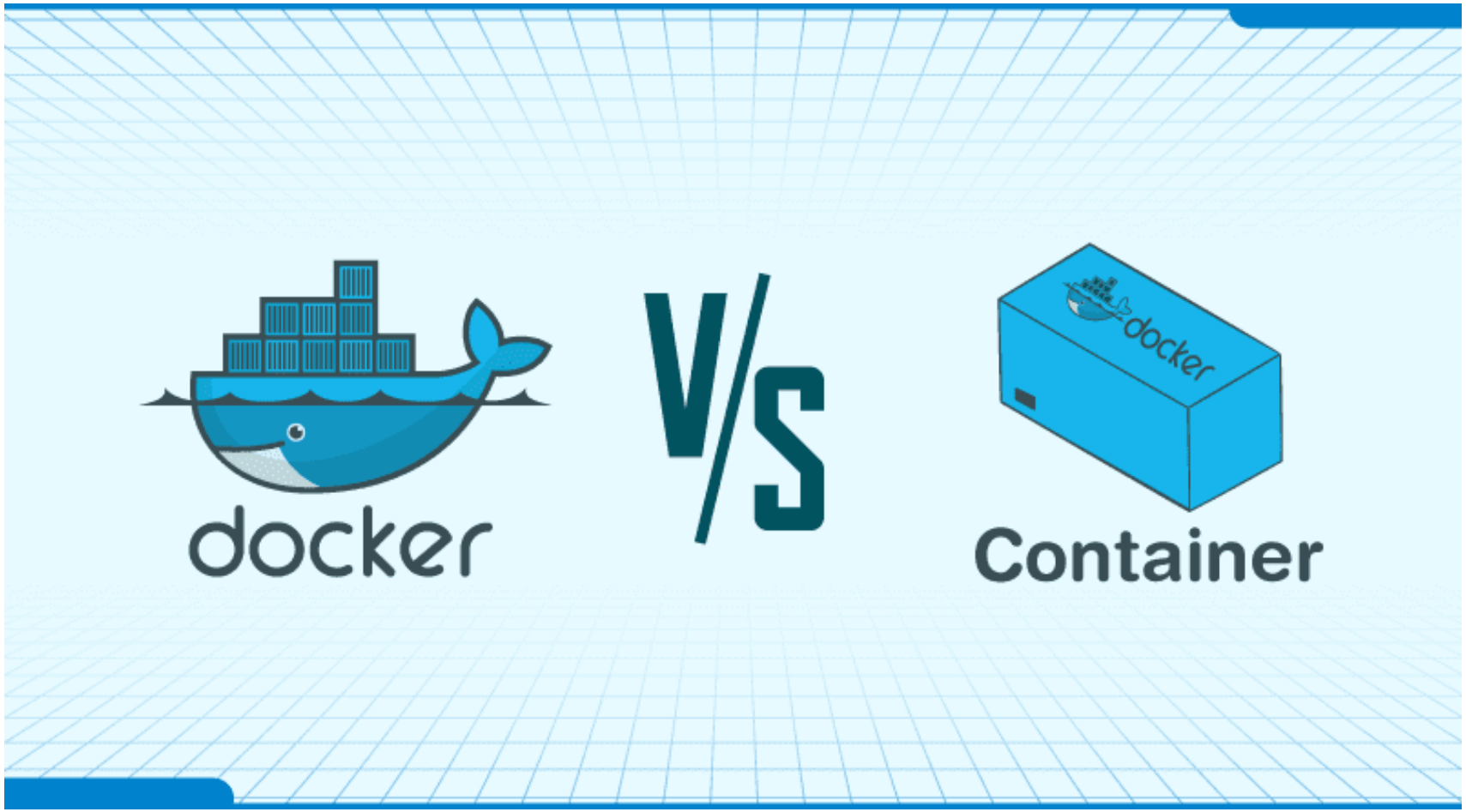
1. Review the product, not the producer
2. Set an agenda and maintain it
3. Limit debate and rebuttal; conduct in-depth discussions off-line
4. Take written notes; utilize a wall board to capture comments
5. Limit the number of participants and insist upon advance preparation
6. Develop a checklist for each product in order to structure and focus the review
7. Allocate resources and schedule time for FTRs
8. Conduct meaningful training for all reviewers
9. Review your earlier reviews to improve the overall review process

DevOps Tools-Docker

- Docker is an open-source project that **automates the deployment of software applications** inside containers by providing an **additional layer of abstraction and automation of OS-level virtualization.**
- It solves the **"works on my machine"** problem by ensuring code runs identically across development, testing, and production environments.
- It allows applications to **share the host OS kernel** instead of **running a separate guest OS like in traditional virtualization.**



DevOps Tools-Docker



DevOps Tools-Docker

Docker Container

- A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another.
- It is a lightweight, standalone, and runnable software package that includes everything needed to run an application, **including code, runtime, system tools, libraries, and settings**
- A Docker container **carries everything an application needs to run except the host operating system kernel.**

- Think of a container as a packaged runtime environment containing:
  - **Our application code** (e.g., Python app, Node.js app, Java application)
  - **Runtime dependencies** (Python interpreter, Node.js runtime, JVM, etc.)
  - **Libraries and packages** required by the application
  - **Configuration files**
  - **Environment variables** (when provided at runtime)
  - **Filesystem contents** needed by the application

DevOps Tools-Docker

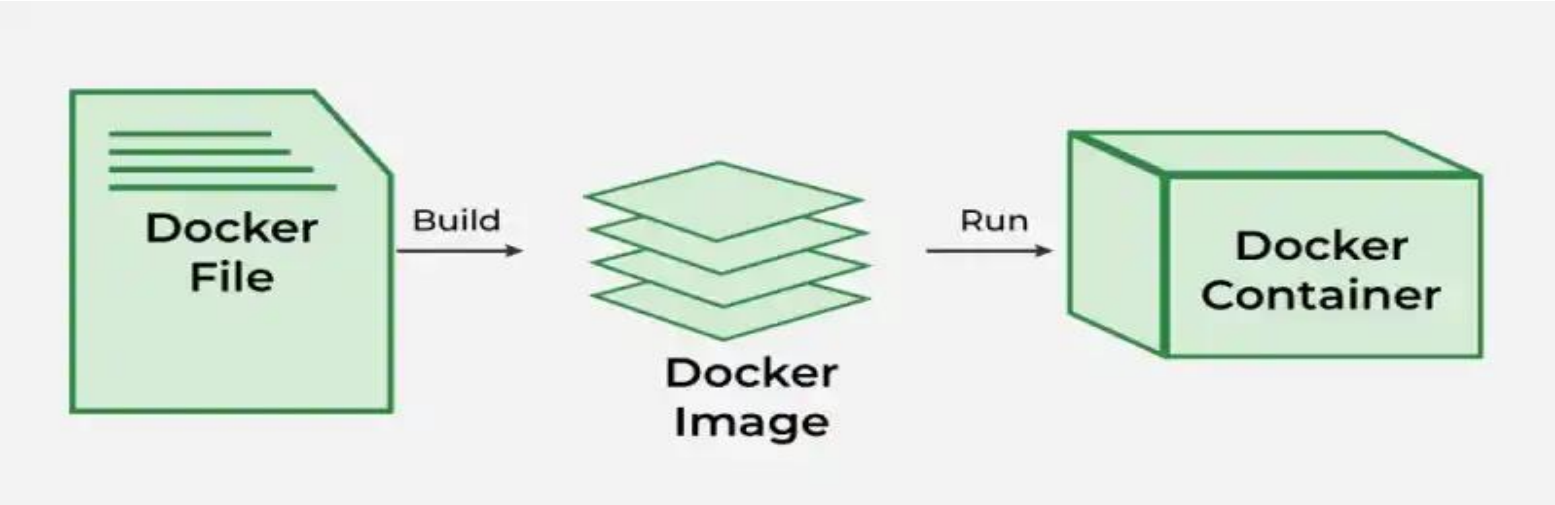
Advantages of Conterization over Virtual Machines

1. Comparatively more light-weighted, resource-efficient and portable
2. **Lower Resource Consumption:** Containers **lack a hypervisor and a guest OS**, removing heavy system overhead. This allows you to **run many more containers on the same physical hardware compared to VMs.**
3. **Instantaneous Startup:** No Guest OS, containers start, scale up and stop fast as compared to VMs.
4. **Ultimate Portability**
5. **Faster CI/CD**

DevOps Tools-Docker

Docker File, Image and Container

- It is a **package or template to create one or more docker container**. Docker images are the fundamental building blocks of containers.
- A Docker Image is a read-only template that contains everything our application needs to run—code, libraries, dependencies, and configurations.
- Images are built from **a set of instructions written in a Dockerfile.**



DevOps Tools-Docker

Docker Engine

- It is the core of Docker, which creates and run containers on the host OS.
- Without the Docker Engine, Docker images cannot be built or containers can't executed.
- Key Components:

1. **Docker Daemon:** Background service that manages containers, images, networks, and volumes.
2. **Docker CLI(Command Line Interface):** command-line tool used to interact with the daemon.
3. **REST API:** allows programs and tools to communicate with Docker.
4. **Container Runtime:** runs containers using technologies such as containerd and runc

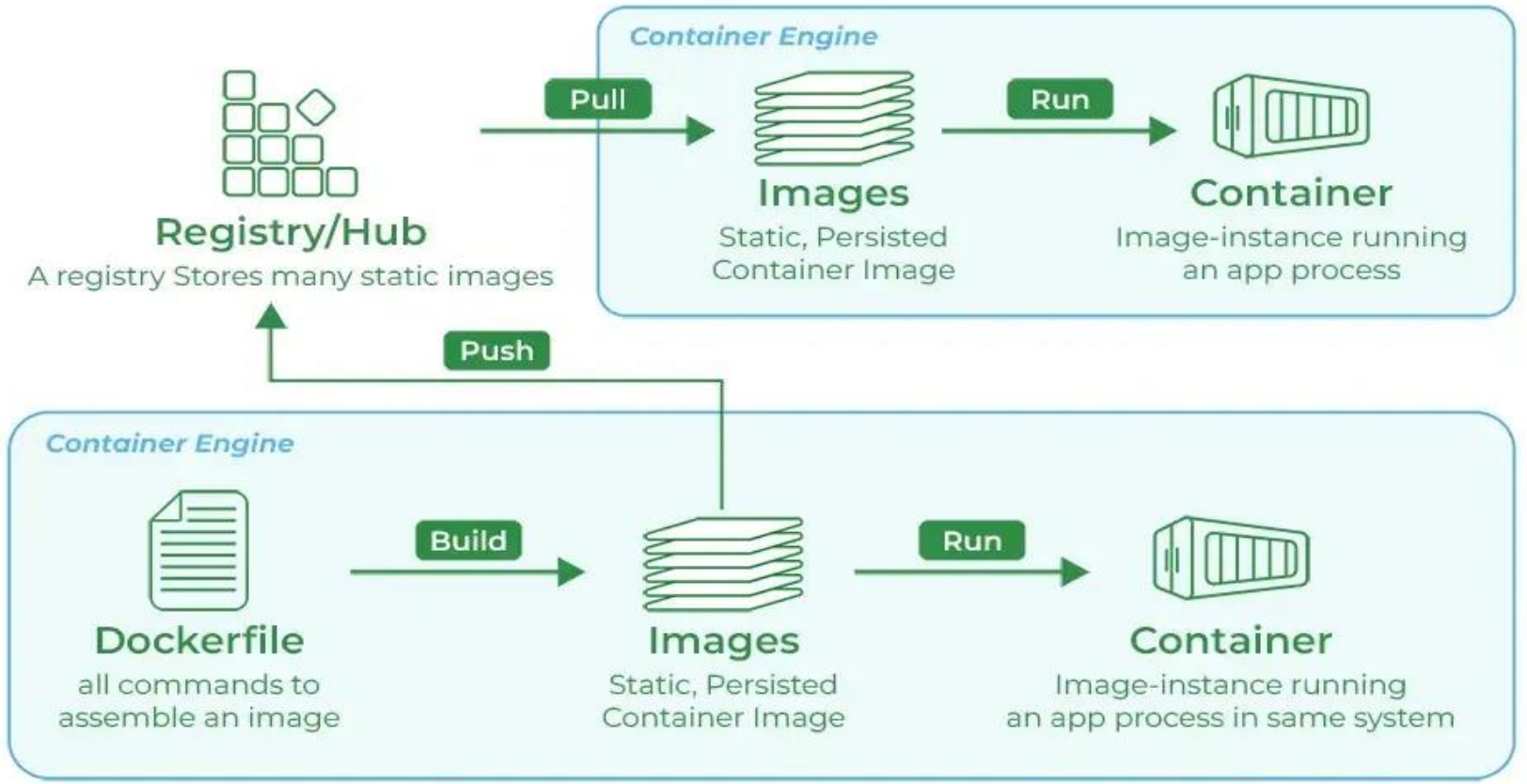
DevOps Tools-Docker

Docker Hub

- Docker Hub is a **cloud-based repository service** where developers and teams can push (upload) and pull (download) Docker container images anytime via the internet.
- It allows images to be stored as public or private, acting like centralized storage that ensures accessibility and version control.
- Developers can create Docker images with all dependencies included, push them to Docker Hub, and make them available for testing or deployment.
- The testing team can directly pull the same image, eliminating the need for additional files, software, or plugins, since the image already contains everything required to run.

DevOps Tools-Docker

Docker Hub



DevOps Tools-Docker

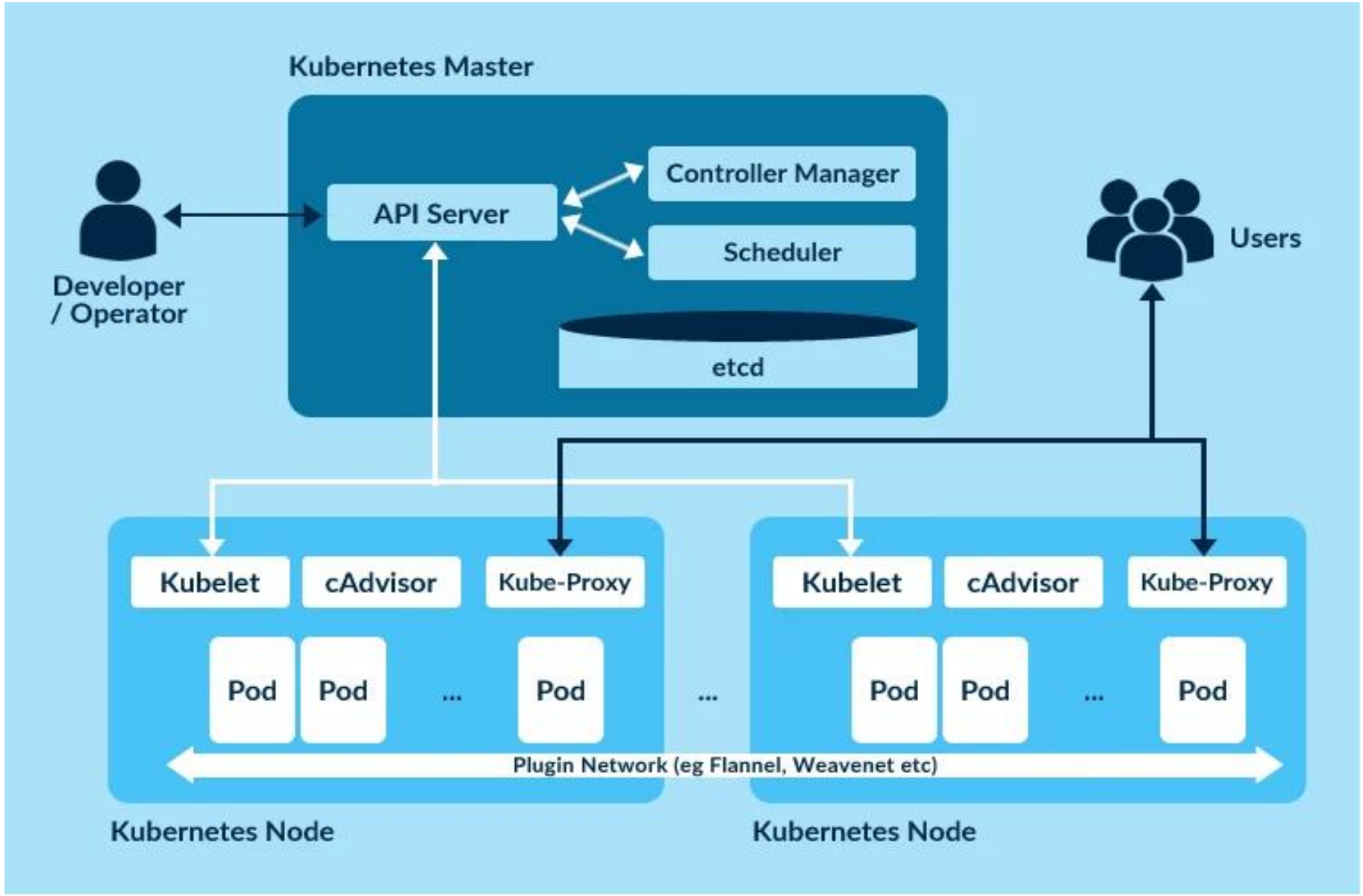
Workflow

1. **Setup:** Download and install Docker Desktop
2. **Create a Docker File:** Create a text file containing the exact instructions.
3. **Build Docker Image:** Compile the docker file created in step 2 and get an executable blueprint called an Image.
4. **Run the Docker Image:** An active container is simply a running instance of your built image.
5. **Manage your containers:** Once your container is running, use these essential CLI commands to manage its lifecycle.
6. **Share your Image:** It is an optional step where we can distribute the image into private or public registry like **Docker Hub.**

DevOps Tools-Kubernetes

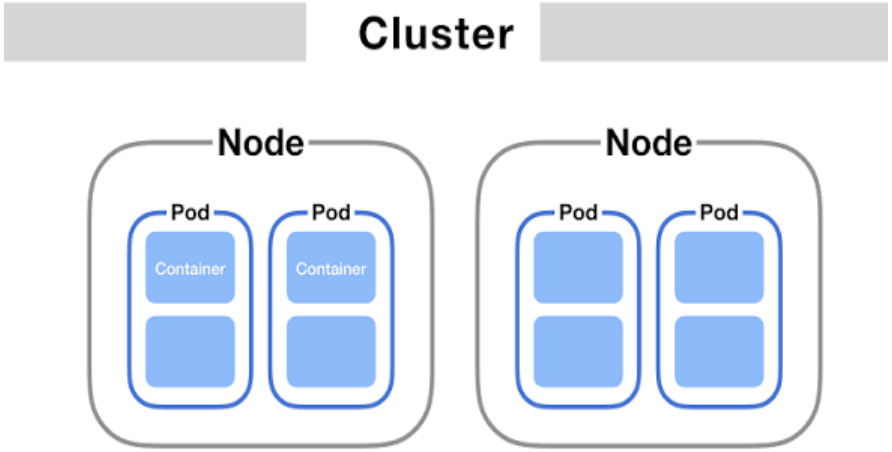
- Kubernetes is an open source platform that helps organizations **manage containerized applications with ease.**
- With several containers operating with different settings, containerized applications may change dynamically and can become complex.
- In the face of dynamically changing workloads, administrators must maintain seamless **orchestration** (the art and practice of writing or adapting music for an orchestra), monitoring, and security while providing scalability, dependability, and resource efficiency.
- This is done by Kubernetes.

DevOps Tools-Kubernetes



DevOps Tools-Kubernetes

1. **Cluster:** A Kubernetes cluster is a **group of computing nodes,** or worker machines, **that run containerized applications.** It is a set of node machines (physical or virtual) working together as a single unified system to run containerized applications. By combining multiple machines, the cluster acts as a single, powerful computer that abstracts away underlying hardware complexities.
2. **Nodes:** Are virtual or physical machines deployed by Kubernetes, i.e., **one machine in your cluster.** There are two types of nodes: A master node and a worker node within a cluster.
3. **Pods:** Pods are the smallest deployable unit present within a Kubernetes cluster. Each node contains a pod that contains one or more containers. They provide the storage, network, and specifications to run containers.



DevOps Tools-Kubernetes

4. **API Server:** is the central management nerve center of a Kubernetes cluster, exposing the HTTP REST API that allows users, internal components, and external plugins to communicate. It **serves as the cluster's front-end gateway.**
5. **Scheduler:** Selects which node an unscheduled pod will run on, based upon the resource availability. It is the duty of the scheduler to make sure that the workload is not scheduled in excess of the available resources.
6. **Controller Manager:** The Kubernetes Controller Manager is a core control plane component that **acts as the "brain" or "automation control center" of a cluster**, running multiple background control loops to **continuously drive the cluster's actual state toward the user-defined desired state.**
7. **etcd: etcd** is the strongly consistent, distributed key-value store that serves as the **primary database and single source of truth** for your entire Kubernetes cluster. It stores the complete configuration data, active state, and metadata of every resource in the cluster.
8. **Kubelet:** Communicates with the master node, gets the list of scheduled containers, and ensures these containers are working systematically.

DevOps Tools-Kubernetes

9. **Kubeproxy:** Ensures pods can communicate with each other within and outside the cluster and forwards traffic to the right IP and port.
10. **Container Runtime:** Responsible for running containers (Docker is a popular container runtime).
11. **CoreDNS:** CoreDNS is the default, highly extensible internal DNS server used for service discovery in Kubernetes clusters.

DevOps Tools-Kubernetes

End-to-end flow of Kubernetes

1. First the developer creates a YAML/JSON file according to what user wants. This is considered as desired state.
2. Developer sends requests to Kubernetes API Server. Sends an HTTP request.
3. API Server validates the request. If valid, the request continues.
4. API Server stores state in etcd in key-value pair.
5. The controller manager then reacts. Deployment Controller detects new deployment. As per the requirement the controller further creates Pods. **Note: Pods are only created but still not assigned to node.**
6. Scheduler then finds best node based on factors like CPU availability, memory availability, node affinity, pod affinity, etc. Also it updates the API Server.

Till now the Kubernetes Master is involved

DevOps Tools-Kubernetes

End-to-end flow of Kubernetes

7. On each worker node Kubelet watches API Server. It sees assigned Pod. It then calls container runtime.
8. Container runtime finally runs containers. It pulls image (Docker image), creates container and finally starts container.
9. Now the Pod becomes running. Kubelet reports back to API Server. Finally the status of Pods is changed to running.
10. Now users can access the Apps using Kubeproxy.

DevOps Tools-Jenkins

- Jenkins is an open-source tool that automates various stages of the software development lifecycle.
- It is an open source CI/CD automation tool that supports the construction, testing, and optimization processes of applications but also automates the entire procedure from code commit to production deployment.
- Its design is based on a master-agent model.

1. **Jenkins Master**: This is the main control unit that runs the build process. It schedules jobs, sends builds to agents, and checks their progress.
2. **Jenkins Agents (Slaves)**: These are separate machines that carry out jobs from the master. They can work on different systems like Windows or Linux.
3. **Job Configuration**: We define jobs as a series of steps that Jenkins will run. We can set them up using the user interface or with configuration files.
4. **Plugins**: Jenkins has a flexible design. This means we can add many plugins. These plugins help us with notifications, version control, build tools, and more.

DevOps Tools-Jenkins

1. **Code Push**: A developer pushes code to a Git repository (e.g., GitHub, GitLab).
2. **Trigger**: Jenkins is triggered via webhook or polling mechanism.
3. **Build**: Jenkins checks out the code and builds the application using tools like Maven, Gradle, or npm.
4. **Test**: Unit tests are run, and results are reported.
5. **Build Docker Image & Push to Registry**: Jenkins builds a Docker image and pushes it to a container registry like Docker Hub or Amazon ECR.
6. **Deploy to Staging**: Jenkins deploys the image to a staging environment using tools like Helm, kubectl, or Ansible.
7. **Manual Approval:** A manual approval step (input block in Jenkinsfile) is used before promoting to production.
8. **Deploy to Production**: After approval, Jenkins deploys the application to the production environment.
9. **Notifications**: Teams are notified via Slack, email, or Jira integration on the pipeline outcome.

Assignment

- Study about other Agile frameworks:
  - eXtreme Programming (XP)
  - Lean Software Development
- Other software process models:
  - RAD (Rapid Application Development)
  - V&V Model
  - Build and Fix Model
  - Aspect Oriented Software Engineering
  - Risk Driven Software Engineering